Memetic Computing (2025) 17:22
https://doi.org/10.1007/s12293-025-00460-8

REGULAR RESEARCH PAPER

A novel multi-agent architecture based on decomposition and learning automata to hybridize multi-objective metaheuristics

Islame F. C. Fernandes[1] · Elizabeth F. G. Goldbarg[2] · Silvia M. D. M. Maia[2]

Received: 1 November 2023 / Accepted: 12 April 2025

Abstract
Hybrid metaheuristics can effectively tackle multi-objective optimization problems. Recently, researchers gained interest in procedures, referred to as *architectures*, that can provide generic functionalities and features for hybridizing arbitrary metaheuristics. Although a previously proposed multi-agent architecture, *MO-MAHM*, achieved high-quality solutions for bi-objective problems, its application for more than two objectives requires further discussions. To this end, *MO*-MAHM$_E$, a *MO-MAHM* extension for handling two or more objectives, is proposed in this paper. *MO*-MAHM$_E$ maintains concepts from particle swarm optimization and multi-agent paradigms, including particle movement and agent intelligence. Further, it uses a decomposition-based velocity operator prescinding from aggregation functions and reinforcement learning automata scheme for supporting the decisions of the agents. This paper shows that these algorithmic components can significantly improve the architectural performance. We apply *MO*-MAHM$_E$ to the quadratic assignment problem with up to four objectives. Hybridization combines three evolutionary algorithms and a local search. A comparison of the experimental results of *MO*-MAHM$_E$ and ten algorithms (including hybrid approaches, hyper-heuristics, and algorithms from the quadratic assignment problem literature) is presented.

Keywords Hybridization of metaheuristics · Multi-objective optimization · Learning automata · Decomposition

1 Introduction

Hybrid metaheuristics combine the best features of individual algorithms. Metaheuristic hybridization has become an attractive research field in single- and multi-objective combinatorial optimization because of its potential to produce high-quality solutions. Existing hybrid metaheuristics rely on several design techniques, some of which were reviewed by [4] and [46]. These techniques exploit the potential of individual algorithms by focusing on intensification and diversification.

As exploiting the best features from individual algorithms is a challenging task, researchers have proposed procedures that provide generic functionalities for implementing hybrid metaheuristics. These procedures, referred to as *architectures*, are general-purpose algorithms for hybridizing arbitrary metaheuristics and have recently gained the interest of researchers [32]. [44] surveyed and analyzed existing architectures. Most of these studies focused on single-objective optimization and employed multi-agent concepts, including agent intelligence and cooperation. In these architectures, each agent implements search methods (e.g., metaheuristics) to solve the problem, and multiple agents can simultaneously explore different regions of the search space. Agents improve their behavior by learning from past experiences, and strengthen the metaheuristic interactions by exchanging search information. Although multi-agent systems benefit the performance of hybrid algorithms, few existing multi-objective optimization architectures fully exploit such concepts [44].

[43] and [13] proposed a multi-objective hybridization architecture, *MO-MAHM*, inspired by Particle Swarm Optimization (*PSO*) and multi-agent concepts, wherein particles are intelligent, autonomous, and cooperative agents that execute actions to obtain near-optimum solutions. Agents employ a learning strategy to improve their behavior and

✉ Islame F. C. Fernandes
islame.felipe@ufba.br

[1] Institute of Computing, Federal University of Bahia (UFBA), Ondina, Salvador, Brazil

[2] Department of Informatics and Applied Mathematics, Federal University of Rio Grande do Norte (UFRN), Campus Universitário, Lagoa Nova, Natal, Brazil

Published online: 17 May 2025

choose appropriate actions for execution. The agents implemented by [13] used the hypervolume indicator [58] in the learning strategy, such that the action that has achieved a higher hypervolume in previous iterations is more likely to be executed in the current iteration. However, [13] observed that the learning strategy in *MO-MAHM* caused the particles to execute certain actions and neglect others consistently. In addition, the agents in *MO-MAHM* execute a search procedure that mimics the velocity operator from the *PSO* to move in the search space. The velocity operator presented by [13] uses the Pareto dominance-based path-relinking metaheuristic to explore the search space between two sets of non-dominated solutions, building trajectories between them. Nevertheless, the authors observed that the velocity operator in *MO-MAHM* tends to lose selection pressure on problems related to a large Pareto-front, draining the agents to small regions of the search space and negatively affecting solution quality and diversity [6]. Despite the high-quality results that *MO-MAHM* has achieved for bi-objective problems, its application to more than two objectives remains an open topic.

This paper proposes a *MO-MAHM* extension, named *MO*-MAHM$_E$, to deal with more than two objectives. Referred to as *many-objective*, optimization problems with more than two objective functions hinder optimizers from ensuring an appropriate balance between diversification and intensification, because the number of non-dominated solutions grows exponentially with the number of objective functions [6, 21]. Several algorithms have been developed for many-objective optimization [30, 40, 51], including metaheuristics such as evolutionary algorithms [2, 27], swarm optimization [26, 39], local search [20, 33], path-relinking [12], and artificial bee colonies [49, 55]. Although previous works have made significant efforts to extend traditional metaheuristics to the many-objective context, none of them have provided generic functionalities for implementing hybrid metaheuristics for many-objective optimization. So, there is still a lack of multi-agent architectures to hybridize these algorithms. Instead of proposing a novel many-objective metaheuristic, this paper proposes an architecture (*MO*-MAHM$_E$) that provides generic procedures to hybridize self-contained many-objective metaheuristics in a multi-agent system.

MO-MAHM$_E$ utilizes a decomposition-based velocity operator alongside a reinforcement learning scheme. Recent advancements in many-objective optimization have integrated decomposition and reinforcement learning techniques [11, 55]. Decomposition methods are primarily favored for addressing many-objective problems, as they effectively balance intensification and diversification [16]. Specifically, *MO*-MAHM$_E$ incorporates a decomposition-based velocity operator derived from the path-relinking metaheuristic [12], which divides the problem into sub-problems without requiring aggregation functions and optimizes them collaboratively. Reinforcement learning systems have significantly enhanced many-objective metaheuristics [17, 18, 35, 38, 50, 54, 56]. These systems consist of agents that interact with an environment, perform actions, and receive feedback that can either be positive or negative. The primary goal of reinforcement learning is to train these agents to maximize the expected cumulative positive feedback. In *MO*-MAHM$_E$, a *learning automata* (*LA*) scheme is employed to determine the transition probabilities for each action pair, which are computed based on the reinforcement signals (positive or negative).

This study applies *MO*-MAHM$_E$ to the multi-objective quadratic assignment problem (*MoQAP*) to evaluate the performance of the proposed architecture. *MoQAP* is a NP-hard facility-location problem [42] that arises in many practical applications. Because the most efficient hybrid multi-objective algorithms in the literature combine evolutionary and local search metaheuristics [46], this work investigates the hybridization of two many-objective evolutionary algorithms [19, 52], a *MoQAP* transgenetic algorithm [1], and a many-objective local search [20]. Experiments were performed on benchmark instances with up to four objectives and different difficulty levels. The analyses considered the benefits of decomposition-based velocity operator and learning strategy. Further, the analyses considered the effect of the cooperation of particles and the contribution of each algorithm hybridized. We compared *MO*-MAHM$_E$ results with those obtained by ten algorithms from the literature. Comparisons involved the original architecture (*MO-MAHM*), three many-objective memetic algorithms (i.e., evolutionary algorithms hybridized with local search), and two hybrid transgenetic algorithms [1]. Comparisons include two state-of-the-art *LA*-based hyper-heuristics proposed by [29] because hyper-heuristics are frameworks for hybridizing algorithms: *HH-LA* and *HH-RILA*. Finally, *MO*-MAHM$_E$ results were compared with two algorithms from the *MoQAP* literature: the *GRASP* proposed by [28] and decomposition-based local search proposed by [5]. Statistical tests showed that *MO*-MAHM$_E$ found better results on most comparisons concerning hypervolume [58] and inverted generational distance indicators [7].

In summary, the main contributions of this paper are as follows:

1. A multi-agent architecture, *MO*-MAHM$_E$, that provides generic procedures to hybridize metaheuristics for problems with three or more objective functions.
2. Results regarding the effect of a decomposition-based velocity operator and an *LA*-based learning strategy on the architecture.
3. An efficient hybrid multi-agent algorithm to the *MoQAP* state-of-the-art, that produces competitive results compared with ten algorithms from the literature.

The remainder of this paper is organized as follows: Section 2 presents the *MO*-MAHM$_E$ architecture. Section 3 presents a *MO*-MAHM$_E$ application for *MoQAP*. Section 4 reports and discusses computational experiments. Section 5 presents conclusions and future work.

2 *MO*-MAHM$_E$: A multi-agent architecture based on Decomposition and Learning Automata for the hybridization of multi-objective metaheuristics

This section details *MO*-MAHM$_E$. Sections 2.1, 2.2, 2.3, and 2.4 review multi-objective concepts, present the *MO*-MAHM$_E$ general procedure, discuss a decomposition-based velocity operator, and present an *LA*-based learning strategy, respectively.

2.1 Multi-objective optimization

This study considers a multi-objective problem defined as

$$\min_{x \in X}(f_1(x), ..., f_q(x)) \quad (1)$$

where $q > 1$ is the number of objectives, X is the set of feasible solutions, $f : X \rightarrow Y$ is a function that assigns to each $x \in X$ an objective vector $f(x) = (f_1(x), ..., f_q(x)) \in Y$, and $Y \subseteq \Re^q$ is the objective space.

"Minimization" considers the Pareto dominance relationship. An objective vector $y = f(x)$, $x \in X$, dominates another $y' = f(x')$, $x' \in X$, denoted by $y \prec y'$, if and only if, $f_k(x) \leq f_k(x')\ \forall k \in \{1, ..., q\}$ and $\exists k \in \{1, ..., q\}$ such that $f_k(x) < f_k(x')$. Pareto dominance concepts extend to X, such that $x \prec x' \Leftrightarrow y \prec y'$. A solution $x \in X$ is efficient if and only if $\nexists x' \in X$ such that $x' \prec x$. The set $X^* \subseteq X$ of efficient solutions is called the Pareto-optimal set. The set $Y^* = \{f(x) \mid x \in X^*\}$ is called the Pareto-optimal front. The set $Y^{*'} \subseteq Y$ is an approximation of the Pareto-optimal front if and only if $\forall y, y' \in Y^{*'}, y \not\prec y'$ and $y' \not\prec y$.

2.2 *MO*-MAHM$_E$ general procedure

MO-MAHM$_E$ is a *MO-MAHM* extension to handle more than two objective functions. *MO-MAHM* was introduced by [13] to bi-objective optimization, and it explored particle swarm optimization (*PSO*) concepts, including cooperation and agent intelligence. *MO-MAHM* maintains a set of cooperative particles that move in the environment, searching for high-quality positions. Each particle is an autonomous agent whose position is a set of non-dominated solutions. Each agent knows its current position, the best position achieved so far (*pbest*), and the best position achieved by the swarm (*gbest*).

Action, learning, and decision strategies are available for each agent. The action strategies are algorithms used to search for efficient solutions. The learning strategies enable agents to improve their behavior by learning the best actions. Learning occurs by extracting information from the environment during the architecture execution. Agents use the knowledge acquired for selecting an action via a method referred to as decision strategy. Further, agents can choose between staying where they are or moving to another position using a velocity operator. The latter is a procedure that receives two or more sets of non-dominated solutions (positions) and outputs another.

Algorithm 1: General *MO-MAHM* procedure [13]

```
   Input: #numPart : Number of particles
1  AS ← loadActionStrategies()
2  DS ← loadDecisionStrategies()
3  LS ← loadLearningStrategies()
4  gbest ← ∅
5  Initialize a set P with #numPart particles
6  repeat
7  |   forall the p ∈ P do
8  |   |   runParticles(p)
9  |   end
10 until A stopping criterion is satisfied;
11 return gbest
```

Algorithm 1 summarizes the *MO-MAHM* procedure presented by [13]. The algorithm receives #*numPart*, which is a user-defined parameter for the number of particles. Lines 1, 2, and 3 represent the load action, decision, and learning strategies, respectively. Line 4 initializes *gbest* as an empty set. Line 5 creates a set with #*numPart* particles and initializes their respective current position and *pbest* attributes. The loop between lines 6 and 10 executes each particle procedure until it satisfies a user-defined stopping criterion. Line 11 outputs *gbest*.

Algorithm 2: *runParticle* procedure [13]

```
   Input: p : Particle
1  l_s ← p.selectLearningStrategy()
2  d_s ← p.selectDecisionStrategy()
3  a_s ← p.selectActionStrategy(l_s, d_s)
4  p.current_position ← a_s.runActionStategy()
5  update p.pbest
6  update gbest
7  dest ← chooseDestination()
8  p.current_position ← velocityOperator(p.current_position, dest)
9  p.runLearningStrategy()
```

Algorithm 2 summarizes the particle procedure proposed by [13]. Lines 1 and 2 select a learning strategy l_s and decision strategy d_s, respectively. Line 3 uses l_s and d_s to choose an action strategy a_s. Line 4 executes a_s, and the non-dominated set outputted becomes the current position of the particle. Lines 5 and 6 use the current position of the new particle to update $pbest$ and $gbest$, respectively. Line 8 moves the particle from its current position towards a destination selected in line 7. Line 9 updates the learning strategy.

MO-MAHM is a flexible framework because there is no assumption about actions, learning, decision strategies, velocity operator, and whether all particles implement the same mechanisms. *MO*-MAHM$_E$ inherits *MO-MAHM* characteristics and follows its general procedure (Algorithms 1 and 2). The novelty in *MO*-MAHM$_E$ is the decomposition-based velocity operator and LA-based learning strategy, as discussed in the following sections. According to the taxonomy presented by [46], *MO*-MAHM$_E$ hybridize self-contained algorithms that cooperate like a teamwork system for finding near-optimum solutions. These characteristics differ *MO*-MAHM$_E$ from standard many-objective optimizers [57].

2.3 Decomposition-based velocity operator

The proposed velocity operator uses a decomposition-based path-relinking procedure [12]. Path-relinking is a metaheuristic introduced by [14] in the single-objective context to explore trajectories between two solutions, s and t. The trajectory from s to t is denoted by $s = x(1), ..., x(i), ..., x(r) = t$, where $x(i)$, $1 < i < r$, represents an intermediate solution, and r represents the trajectory length. A path generation procedure computes intermediate solutions and obtains $x(i+1)$ from $x(i)$'s neighborhood, and $1 \leq i < r$, such that $x(i+1)$ is more similar to t than $x(i)$. The velocity operator relies on decomposing the objective space and leads to an appropriate balance between the quality and diversity of solutions. Let $N = \{\gamma^1, ..., \gamma^{|N|}\} \subset \Re^q$, $q \geq 2$ be a set of $|N|$ uniformly distributed non-negative direction vectors, where $|N|$ represents a user-defined parameter. The method decomposes the objective space Y into $|N|$ sub-regions $Y^1, ..., Y^{|N|}$, where Y^j, $j = 1, ..., |N|$, represent the sub-region related to γ^j. No aggregation function is required.

Let A be a set of non-dominated solutions. Equation (2) normalizes every objective vector $f(x)$, $x \in A$, where $LB \in \Re^q$ and $UB \in \Re^q$ are lower and upper bounds, respectively. Equation (3) calculates the cosine of the angle between γ^j and $f_{norm}(x)$, denoted by $\Delta(f(x), \gamma^j)$, where $\|.\|$ is the Euclidean norm. The method divides solutions in A into $|N|$ subsets $S^1, ..., S^{|N|}$, where S^j ($j = 1, ..., |N|$) is the subset of solutions related to sub-region Y^j and contains the solutions for which $\Delta(f(x), \gamma^j)$ is the maximum. There is an ideal point $I^j \in \Re^q$ related to each sub-region Y^j. I^j is the point with a minimal distance to the origin along the direction vector γ^j [39].

$$f_{norm}(x) = \frac{f(x) - LB}{UB - LB} \tag{2}$$

$$\Delta(f(x), \gamma^j) = \frac{\gamma^j \cdot f_{norm}(x)}{\|\gamma^j\| \|f_{norm}(x)\|}, \; j = 1, ..., |N| \tag{3}$$

Minimizing the Euclidean distance from $f_{norm}(x)$ to I^j denoted by $\|I^j - f_{norm}(x)\|$ promotes the convergence of solutions in S^j towards I^j. Equation (4) computes the ratio between $\Delta(f(x), \gamma^j)$ and $\|I^j - f_{norm}(x)\|$ to ensure the quality and diversity of solutions, which is denoted by $CAD(x, j)$ [39]. The higher the $CAD(x, j)$, the closer is $f_{norm}(x)$ to Y^j and I^j; therefore, the better is $f(x)$.

$$CAD(x, j) = \frac{\Delta(f(x), \gamma^j)}{\|I^j - f_{norm}(x)\|}, \; j = 1, ..., |N| \tag{4}$$

The decomposition-based velocity operator performs path-relinking trajectories between solutions of two input approximation sets A and B. The set of non-negative direction vectors N is an input parameter. Any method that creates direction vectors uniformly distributed on the objective space can compute N. Ideal points $I^1, ..., I^{|N|}$ are also input parameters. The method first divides solutions in A and B into subsets $S^1, ..., S^{|N|}$. For each $j = 1, ..., |N|$, the velocity operator chooses two solutions $s, t \in S^j$ and generates a path-relinking trajectory $s = x(1), ..., x(r) = t$ guided by $CAD(x(i), j), i = 1, ..., r$. Subsets $S^1, ..., S^{|N|}$ are updated with non-dominated solutions in the trajectories. The velocity operator outputs a set containing at most $|N|$ well-spread solutions selected from $S^1, ..., S^{|N|}$.

Algorithm 3 outlines the decomposition-based velocity operator. Line 1 initializes S^j, $j = 1, ..., |N|$. Lines 2 and 3 distribute solutions in A and B, respectively, into subsets S^j, $j = 1, ..., |N|$ (procedure *distributeSolutions*). Line 4 updates ideal points $I^1, ..., I^{|N|}$ (procedure *computeIdealPoints*). For each $j = 1, ..., |N|$, the algorithm generates a backward trajectory [41] connecting two solutions x' and x'' in S^j. The starting solution is the one with the best CAD (lines 9 and 11). The goal is to promote the search near the neighborhood of the solution with the best CAD. Set Arc stores the non-dominated solutions obtained by the path generation. Line 14 distributes the solutions in Arc into subsets $S^1, ..., S^{|N|}$. Line 15 updates the ideal points. The algorithm uses a variation of the binary tournament to select solutions to output. Line 16 initializes a set C as an empty set. For each $j = 1, ..., |N|$, line 18 selects two solutions x' and x'' in S^j at random, line 19 chooses the one with the best CAD, and line 20 adds it to C without verifying the Pareto dominance. The algorithm outputs C.

Algorithm 3: Decomposition-based velocity operator

Input : A, B: approximation sets
$N = \{\gamma^1, ..., \gamma^{|N|}\}$: Set of direction vectors
I^j: Ideal point related to γ^j, for $j = 1, ..., |N|$
Output: C: Set of solutions

1 $S^j \leftarrow \emptyset, \forall j = 1, ..., |N|$
2 *distributeSolutions*$(A, N, S^1, ..., S^{|N|})$
3 *distributeSolutions*$(B, N, S^1, ..., S^{|N|})$
4 *computeIdealPoints*$(N, S^1, ..., S^{|N|}, I^1, ..., I^{|N|})$
5 $Arc \leftarrow \emptyset$
6 **foreach** $j = 1, ..., |N|$ **do**
7 Let x', x'' be two solutions in S^j
8 **if** $CAD(x', j) > CAD(x'', j)$ **then**
9 *path_generation*(x', x'', j, Arc)
10 **else**
11 *path_generation*(x'', x', j, Arc)
12 **end**
13 **end**
14 *distributeSolutions*$(Arc, N, S^1, ..., S^{|N|})$
15 *computeIdealPoints*$(N, S^1, ..., S^{|N|}, I^1, ..., I^{|N|})$
16 $C \leftarrow \emptyset$
17 **foreach** $j = 1, ..., |N|$ **do**
18 Select two solutions randomly, x', x'', in S^j
19 $\hat{x} \leftarrow \arg\max\{CAD(x, j) \mid x \in \{x', x''\}\}$
20 Add $\hat{x}$ to C
21 **end**
22 **return** C

Algorithm 4: *distributeSolutions* procedure

Input : A: Approximation set
$N = \{\gamma^1, ..., \gamma^{|N|}\}$: Set of direction vectors
S^j: Subset of solutions related to γ^j, for $j = 1, ..., |N|$
Output: S^j: Subset of solutions related to γ^j, for $j = 1, ..., |N|$

1 $\forall x \in A$, x is not marked
2 **foreach** $j = 1, ..., |N|$ **do**
3 $x' \leftarrow \arg\max\{\Delta(f(x), \gamma^j) \mid x \in A$ and x not marked$\}$
4 mark x'
5 add x' to S^j
6 **end**

The *distributeSolutions* procedure in lines 2, 3, and 14 of Algorithm 3 receives a set of non-dominated solutions (say, A), set N, and subsets $S^1, ..., S^{|N|}$. The procedure distributes solutions in A into $S^1, ..., S^{|N|}$. For each $j = 1, ..., |N|$, it adds the solution $x' \in A$ with the greatest $\Delta(f(x'), \gamma^j)$ to S^j such that x' is not selected. The goal is to avoid having duplicate solutions in the subsets. Algorithm 4 outlines the procedure *distributeSolutions*.

The method proposed by [39], referred to as *computeIdealPoints*, computes the ideal points. I^j, $j = 1, ..., |N|$, is the point projected on γ^j with a minimal distance to the origin. Equation (5) defines the scalar projection $\Psi(f(x), \gamma^j)$ of $f_{norm}(x)$ on γ^j, for all $j = 1, ..., |N|$ and $x \in S^j$, where $\|.\|$ is the Euclidean norm. For each $j = 1, ..., |N|$, the procedure identifies the lowest $\Psi(f(x), \gamma^j)$, $x \in S^j$ and assigns it to ψ. I^j is set to $\psi\gamma^j$ if $\psi\gamma^j$ dominates I^j. Algorithm 5 outlines the *computeIdealPoints* procedure.

$$\Psi(f(x), \gamma^j) = \frac{\gamma^j \cdot f_{norm}(x)}{\|\gamma^j\|}, \; j = 1, ..., |N| \tag{5}$$

Algorithm 5: *computeIdealPoints* procedure

Input : $N = \{\gamma^1, ..., \gamma^{|N|}\}$: Set of direction vectors
S^j: Subset of solutions related to γ^j, for $j = 1, ..., |N|$
I^j: Ideal point related to γ^j, for $j = 1, ..., |N|$
Output: I^j: Ideal point related to γ^j, for $j = 1, ..., |N|$

1 **foreach** $j = 1, ..., |N|$ **do**
2 $\psi \leftarrow \min\{\Psi(f(x), \gamma^j) \mid x \in S^j\}$
3 **if** $\psi\gamma^j \prec I^j$ **then**
4 $I^j \leftarrow \psi\gamma^j$
5 **end**
6 **end**

Algorithm 6: *path_generation* procedure

Input : s: Starting solution,
t: Targeting solution,
j: Sub-problem index,
Arc: Set of non-dominated solutions
Output: Arc: Set of non-dominated solutions

1 $i \leftarrow 1$
2 $x(i) \leftarrow s$
3 **while** $M(x(i), t) \neq \emptyset$ **do**
4 $m^* \leftarrow \arg\max\{CAD((x(i) \oplus m), j) \mid m \in M(x(i), t)\}$
5 $x(i+1) \leftarrow x(i) \oplus m^*$
6 Add $x(i+1)$ to Arc and filter Arc, if necessary
7 $i \leftarrow i + 1$
8 **end**

The *path_generation* procedure receives a starting solution s, a targeting solution t, a sub-problem index $j \in \{1, ..., |N|\}$, and an archive Arc to store the non-dominated solutions. The procedure builds a trajectory $s = x(1), ..., x(r) = t$ connecting s and t. We denote the set of candidate operations that can be applied to $x(i)$ to generate an intermediate solution $x(i+1)$ more similar to t than $x(i)$ by $M(x(i), t)$, $i = 1, ..., r-1$. Given $m \in M(x(i), t)$, $x(i) \oplus m$ denotes the application of m to $x(i)$. Every iteration, $i = 1, ..., r-1$, the procedure selects $m^* \in M(x(i), t)$ such that $CAD(x(i) \oplus m^*, j)$ is the maximum and obtains $x(i+1) = x(i) \oplus m^*$. This step can consume extra evaluations to compute the objective function of candidate solutions and the CAD indicator. All function evaluations are calculated in $\#num\_eval$. The next step adds $x(i+1)$ to Arc and removes the dominated solutions if necessary. Algorithm 6 outlines the *path_generation* procedure.

2.4 Learning Strategy

MO-MAHM$_E$ uses an online learning strategy based on the *LA* model implemented by [29]. The authors proposed two *LA*-based multi-objective hyper-heuristics, called *HH-LA* and *HH-RILA*, which maintain a population of solutions and a set of low-level heuristics (actions). The *LA* model computes transition probabilities and reward/penalty rates for each pair of actions using the hypervolume improvement [58] promoted on the current population. In every iteration, *HH-LA* and *HH-RILA* use the transition probability to select a low-level heuristic and apply it to the current population. Experiments reported in [29] showed that *LA* benefits multi-objective selection hyper-heuristics.

This study adapted the model presented by [29] to consider the improvement promoted on *pbest* and *gbest*. Let $(AS, \phi, \mathbb{P}, \mathbb{U})$ be a quadruple, where AS is the finite set of action strategies implemented within the *MO*-MAHM$_E$, ϕ is the reinforcement signal, $\mathbb{P}$ is a transition matrix, and $\mathbb{U}$ is the updating scheme. For all $\hat{a}, \overline{a} \in AS$, there is an entry in matrix $\mathbb{P}$. $\mathbb{P}_{(\hat{a},\overline{a})}(\tau)$ represents the transition probability of particle p switching from $\hat{a}$ to $\overline{a}$ at the τ-th search step. $\phi(\tau)$ denotes the reinforcement signal that p receives from the environment after p applying an action strategy at the τ-th step. $\phi(\tau) = 1$ if the action succeeded in improving *pbest* or *gbest*, and $\phi(\tau) = 0$ otherwise. Quality indicators or techniques suitable for assessing the quality of non-dominated sets can be used to measure the improvement. For all $\hat{a}, \overline{a} \in AS$, $\xi_{(\hat{a},\overline{a})}(\tau) \in [0, 1]$ represents the reward/penalty rate at the τ-th step related to the transition from $\hat{a}$ to $\overline{a}$.

Hereinafter, we denote by a and a' the action strategies p applied at $(\tau-1)$-th and τ-th steps, respectively. Equation (6) updates the probability $\mathbb{P}_{(a,a')}(\tau+1)$. Equation (7) updates $\mathbb{P}_{(a,a'')}(\tau+1)$, for all $a'' \in AS, a'' \neq a'$, where $|AS|$ is the number of action strategies.

$$\mathbb{P}_{(a,a')}(\tau+1) = \begin{cases} \mathbb{P}_{(a,a')}(\tau)(1-\xi_{(a,a')}(\tau)) + \xi_{(a,a')}(\tau) & \text{if } \phi(\tau)=1 \\ \mathbb{P}_{(a,a')}(\tau)(1-\xi_{(a,a')}(\tau)) & \text{if } \phi(\tau)=0 \end{cases} \tag{6}$$

$$\mathbb{P}_{(a,a'')}(\tau+1) = \begin{cases} \mathbb{P}_{(a,a'')}(\tau)(1-\xi_{(a,a'')}(\tau)) & \text{if } \phi(\tau)=1 \\ \mathbb{P}_{(a,a'')}(\tau)(1-\xi_{(a,a'')}(\tau)) + \frac{\xi_{(a,a'')}(\tau)}{|AS|-1} & \text{if } \phi(\tau)=0 \end{cases} \tag{7}$$

For all $\hat{a}, \overline{a} \in AS$, the improvement produced on *pbest* and *gbest* is estimated. $Pb_{(\hat{a},\overline{a})}(\tau)$ and $Gb_{(\hat{a},\overline{a})}(\tau)$ are the accumulated improvement at the τ-th step related to the pair $(\hat{a}, \overline{a})$ on *pbest* and *gbest*, respectively. Equation (8) computes the reward/penalty rate $\xi_{(\hat{a},\overline{a})}(\tau)$, $\forall \hat{a}, \overline{a} \in AS$. User-defined coefficients ν_{pb} and ν_{gb} amplify the effect of $Pb_{(\hat{a},\overline{a})}(\tau)$ and $Gb_{(\hat{a},\overline{a})}(\tau)$, respectively. If $\xi_{(\hat{a},\overline{a})}(\tau) > 1$, then it is set to 1. We use a minimal fixed rate of 0.1 to avoid $\xi_{(\hat{a},\overline{a})}(\tau) = 0$, as suggested by [29].

$$\xi_{(\hat{a},\overline{a})}(\tau) = 0.1 + \nu_{pb} Pb_{(\hat{a},\overline{a})}(\tau) + \nu_{gb} Gb_{(\hat{a},\overline{a})}(\tau) \tag{8}$$

After executing a' at the τ-th step, particle p measures the quality of *pbest* and *gbest*. We denote the quality of *pbest* and *gbest* by $Q_{pb}(\tau)$ and $Q_{gb}(\tau)$, respectively. Without loss of generality, greater Q_{pb} and Q_{gb} values indicate better quality. If $Q_{pb}(\tau) > Q_{pb}(\tau-1)$ after executing a', the difference $\delta_{pb}(\tau) = Q_{pb}(\tau) - Q_{pb}(\tau-1)$ enables the improvement produced by a' on *pbest* at the end of step τ. The $\delta_{gb}(\tau)$ is analogous for $Q_{gb}(\tau)$ and *gbest*. Equations (9) and (10) compute $Pb_{(a,a')}(\tau)$ and $Gb_{(a,a')}(\tau)$, respectively. For all $a'' \in AS, a'' \neq a'$, equations (11) and (12) compute $Pb_{(a,a'')}(\tau)$ and $Gb_{(a,a'')}(\tau)$, respectively. Previous $Pb_{(a,a'')}(\tau-1)$ and $Gb_{(a,a'')}(\tau-1)$ decrease at a fixed rate of 0.1, as suggested by [29]. The goal is to assign less weight to past improvements.

$$Pb_{(a,a')}(\tau) = \begin{cases} 0.9 Pb_{(a,a')}(\tau-1) + 0.1\delta_{pb}(\tau) & \text{if } \delta_{pb}(\tau) > 0 \\ 0.9 Pb_{(a,a')}(\tau-1) & \text{otherwise} \end{cases} \tag{9}$$

$$Gb_{(a,a')}(\tau) = \begin{cases} 0.9 Gb_{(a,a')}(\tau-1) + 0.1\delta_{gb}(\tau) & \text{if } \delta_{gb}(\tau) > 0 \\ 0.9 Gb_{(a,a')}(\tau-1) & \text{otherwise} \end{cases} \tag{10}$$

$$Pb_{(a,a'')}(\tau) = 0.9 Pb_{(a,a'')}(\tau-1) \tag{11}$$

$$Gb_{(a,a'')}(\tau) = 0.9 Gb_{(a,a'')}(\tau-1) \tag{12}$$

3 *MO*-MAHM$_E$ application to *MoQAP*

The multi-objective quadratic assignment problem (*MoQAP*) is an extension of the quadratic assignment problem (*QAP*) for multi-objective optimization. First introduced by [25], the *QAP* is a facility-location problem considering the distance between locations and flow between facilities. The goal is to allocate facilities into locations such that the sum of the products of distances and corresponding flows is minimum. The *MoQAP* models scenarios with $q \geq 2$ flows between each pair of facilities.

Let $\mathbb{A} = (\alpha_{ij})$ be a n-square matrix, where α_{ij} is the distance between locations i and j, $1 \leq i, j \leq n$. There is an n-square matrix $\mathbb{B}^k = (\beta^k_{ij})$ for each objective k, $1 \leq k \leq q$, where β^k_{ij} is the k-th flow from facility i to j. A feasible solution is a permutation π of set $\{1, ..., n\}$, where π_i is the location of the i-th facility. The *MoQAP* minimizes $f(\pi) = (f_1(\pi), ..., f_q(\pi)) \in \Re^q$, where equation (13) defines $f_k(\pi)$, $1 \leq k \leq q$. The problem is symmetric when $\mathbb{A}$ and $\mathbb{B}^k$,

$1 \leq k \leq q$, are symmetric.

$$f_k(\pi) = \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_{ij}\beta^k_{\pi_i\pi_j}, \ \forall k \in \{1, ..., q\} \quad (13)$$

The *MoQAP*, as well as its single-objective counterpart, is an *NP-hard* problem [42]. The existing studies on this topic include heuristics approaches based on GRASP, evolutionary algorithms, local search, and hybrid approaches. Recent studies regarding those approaches are [28] and [5].

Section 3.1 describes the algorithms hybridized within the architecture (action strategies). Section 3.2 presents the *MO-MAHM*$_E$ implementation.

3.1 Action strategies for the *MoQAP*

Action strategies implemented for bi-objective instances were a transgenetic algorithm (*TA*) [1], an *NSGA-II* [10], a *MOEA/D* [53], and a local search (*MPLS*) [20]. For instances with more than two objectives, action strategies are *TA*, *A-NSGA-III* [19], *MOEA/D-DU* [52], and *MPLS*. Because *NSGA-II* and *MOEA/D* are suitable for bi-objective problems and face difficulties in dealing with an increasing number of objectives [19, 52], we implemented their respective state-of-the-art many-objective versions for instances with three or more objectives: (*A-NSGA-III* and *MOEA/D-DU*).

3.1.1 Transgenetic Algorithm

TAs are evolutionary approaches that simulate horizontal gene transfer and endosymbiosis [15]. They employ search mechanisms (called *transgenetic vectors*) for exchanging genetic information between a population of solutions (called *endosymbionts*) and genetic information repository (called *host cell*).

TA proposed by [1] for *MoQAP* maintains a population Pop with $\#popSize$ endosymbionts, where $\#popSize$ represents a user-defined parameter. An endosymbiont is a *MoQAP* feasible solution. *TA* generates the initial population at random and initializes the host's repository with *a priori* and *a posteriori* information. The *a priori* repository stores a solution obtained by the robust Tabu search algorithm [45]. The aspiration criterion is the Tchebycheff function defined by equation (14), where $y^* = (y^*_1, ..., y^*_q)$ is the ideal point, and $\lambda = (\lambda_1, ..., \lambda_q) \in \Re^q$ is a random weight vector satisfying equation (15). The *a posteriori* repository stores $\#sizeRepo$ solutions, where $\#sizeRepo$ is a user-defined parameter. The initial *a posteriori* repository contains the best endosymbionts in Pop based on the sum of objective values.

$$f_{TCH}(\pi|\lambda) = \max_{1\leq k\leq q} \{\lambda_k | f_k(\pi) - y^*_k|\} \quad (14)$$

$$\sum_{k=1}^{q} \lambda_k = 1 \quad (15)$$

TA implements two types of transgenetic vectors: plasmids and transposons. Plasmids include genetic information and a method to insert them into the endosymbionts. Let π be a random solution chosen with uniform probability from the host's repository. The genetic information of the plasmid is a string of random pairs $(i, \pi_i), i \in \{1, ..., n\}$. The length of the genetic information is random and comes from the interval $[3, 3{+}0.25n]$. Let π' be an endosymbiont from Pop. For each pair (i, π_i) in the information string, the plasmid identifies index j such that $\pi'_j = \pi_i$ and swaps π'_i and π'_j.

Transposons are rules for rearranging a restricted fragment of an endosymbiont. The transposon implemented in *TA* uses the robust Tabu search [45]. The method explores the neighborhood of a continuous fragment of the endosymbiont using the Tchebycheff function. The length of the fragment is random and comes from the interval $[\frac{n}{2}, n]$.

Transgenetic vectors manipulate endosymbionts in every iteration. The user-defined parameter $\#probPlasm$ is the probability of applying the plasmid; the probability of applying the transposon is $1 - \#probPlasm$. The value of $\#probPlasm$ remains fixed for $\#intGerSet$ consecutive iterations, and then, it decrements by a factor $\#probFactor$. $\#intGerSet$ and $\#probFactor$ are user-defined parameters. In this study, *TA* maintains a global archive of non-dominated solutions G_A. An endosymbiont π'' resulting from a plasmid or transposon manipulation replaces the original endosymbiont π' in Pop if $\pi'' \prec \pi'$ or if $\nexists \pi'''$ in G_A such that $\pi''' \prec \pi''$. The procedure adds π'' to G_A and removes dominated solutions. If π'' is better, according to the sum of objective values, than the worst endosymbiont in the *a posteriori* repository then π'' replaces it.

3.1.2 NSGA-II and A-NSGA-III

Non-dominated sorting genetic algorithm II (*NSGA-II*) [10] maintains a population Pop with $\#sizePop$ solutions. In every iteration, the algorithm applies genetic operators (mating selection, recombination, and mutation) for creating an offspring population $\overline{Pop}$ with $\#sizePop$ individuals. The procedure sorts solutions from $Pop \cup \overline{Pop}$ into different levels (fronts) of non-domination. The first front F_1 contains non-dominated solutions from $Pop \cup \overline{Pop}$, the second front F_2 those from $Pop \cup \overline{Pop} - F_1$, and so on.

The population for the next iteration comprises $\#sizePop$ solutions selected from those fronts according to the sequence $F_1, F_2, ...$ If the l-th front (F_l) insertion exceeds $\#sizePop$

individuals, then *NSGA-II* selects the solutions with the largest crowding distance metric from F_l to complete the next population. The crowding distance of a solution measures the distance between its two nearest neighbors in objective space.

NSGA-II proved to be effective for bi-objective problems. However, it produces low-quality solutions for more than two objectives because of the exponential number of non-dominated solutions and expensive cost of computing the crowding distance [9]. *A-NSGA-III* handles these difficulties by selecting individuals from F_l according to a niching scheme defined by a set $W \subseteq \Re^q$ of $\#sizePop$ reference points introduced by [19]. The first step normalizes the objective vectors; the second step associates individuals from $F_1, ..., F_l$, with the closest reference point concerning the perpendicular distance. Equation (16) defines the perpendicular distance $d^{\perp}(\pi|w)$ from a solution π to a reference point $w \in W$. The procedure iteratively inserts into the next population a solution from F_l associated with the reference point with the minimum number of associations. The last step verifies whether there exist reference points associated with no solution. Because such "non-useful" reference points do not contribute to the search and deteriorate the algorithm performance, *A-NSGA-III* removes them and generates new ones.

$$d^{\perp}(\pi|w) = \| f(\pi) - w \left(\frac{f(\pi)^T w}{\|w\|^2} \right) \| \tag{16}$$

In this study, we implemented an *NSGA-II* for bi-objective *MoQAP* instances and *A-NSGA-III* for more than two objectives. The initial population consists of random solutions. In every iteration, $\#rateCross$ and $\#rateMut$ parameters define the probabilities of applying crossover and mutation operators, respectively. We used the one-point crossover and shift mutation presented by [37]. The crossover chooses a random point i and copies the first i values of the first parent to the offspring solution. For each value of the second parent, the procedure copies it to the child when the value is not in the offspring solution. The shift mutation chooses two random positions of solution π (say i' and i''), inserts $\pi_{i'}$ in position i'', and shifts the other elements left by one position.

3.1.3 MOEA/D and MOEA/D-DU

The decomposition-based multi-objective evolutionary algorithm (*MOEA/D*) introduced by [53] maintains a population of solutions Pop and a set of weight vectors $\Lambda \subseteq \Re^q$. The user-defined parameter $\#popSize$ indicates the number of elements in Pop and Λ. Solution $\pi^j \in Pop$, $j = 1, ..., \#popSize$, is the current solution to the j-th subproblem defined by $\lambda^j = (\lambda^j_1, ..., \lambda^j_q) \in \Lambda$. The j-th subproblem minimizes the Tchebycheff function $f_{TCH}(\pi|\lambda^j)$. $\aleph(j)$ is the set of $\#neigb$ closest weight vectors to λ^j concerning the Euclidean distance. In every iteration $j = 1, ..., \#popSize$, the standard *MOEA/D* randomly selects two indexes j' and j'' from $\aleph(j)$, and genetic operators generate an offspring solution $\overline{\pi}$ from $\pi^{j'}$ and $\pi^{j''}$.

For each $j''' \in \aleph(j)$, if $f_{TCH}(\overline{\pi}|\lambda^{j'''}) \leq f_{TCH}(\pi^{j'''}|\lambda^{j'''})$, then $\overline{\pi}$ replaces $\pi^{j'''}$ in Pop.

Although *MOEA/D* produces satisfactory results for bi-objective problems, it presents a poor balance between diversity and convergence for problems with more than two objectives [52]. *MOEA/D-DU* is a *MOEA/D* extension proposed by [52] for handling more than two objectives. The algorithm computes the difference $\| f(\pi) - y^* \|$ for all π in the population, where y^* is the ideal point. In every iteration $j = 1, ..., \#popSize$, genetic operators create an offspring solution $\overline{\pi}$ from π^j and $\pi^{j'}$, where j' is selected from $\aleph(j)$ or from $\{1, ..., \#popSize\}$. The user-defined parameter $\#probNeigb \in [1, 0]$ is the probability of using $\aleph(j)$. The probability of using $\{1, ..., \#popSize\}$ is $1 - \#probNeigb$. The algorithm computes the perpendicular distance $d^{\perp}(\overline{\pi}|\lambda)$ defined in equation (16) for each $\lambda \in \Lambda$ and selects the $\#maxReplc$ minimum distances, where $\#maxReplc \leq \#popSize$ is a user-defined parameter. If there exists $\lambda^{j''}$, j'' ranges from 1 to $\#maxReplc$, such that $f_{TCH}(\overline{\pi}|\frac{1}{\lambda^{j''}}) < f_{TCH}(\pi^{j''}|\frac{1}{\lambda^{j''}})$, then $\overline{\pi}$ replaces $\pi^{j''}$ in Pop.

In this study, we implemented *MOEA/D* for *MoQAP* with two objectives, and *MOEA/D-DU* for more than two objectives. We used the initial population, crossover, and mutation operators described in section 3.1.2.

3.1.4 MPLS

Pareto local search (*PLS*) is a local search technique based on Pareto dominance for multi-objective combinatorial optimization problems. Given an initial set of non-dominated solutions S, *PLS* iteratively selects a random solution $\pi \in S$, explores the neighborhood of π, updates S with the non-dominated neighbors of π, and removes dominated solutions from S. The algorithm stops after exploring all solutions in S. In addition to requiring an expensive computational effort for exploring all solutions, *PLS* presents a poor anytime behavior, i.e., it does not guarantee a high-quality S at any time during its execution.

The *PLS* variant proposed by [20] (*MPLS*) handles two or more objectives and improves the search anytime behavior. The authors address new mechanisms to select $\pi \in S$, explore neighborhoods of π, and update S. In every iteration, *MPLS* generates a random weight vector $\lambda \in \Re^q$ satisfying (15) and selects $\pi \in S$ such that $f_{TCH}(\pi|\lambda)$ is the minimum. The algorithm explores $\#maxNeigb$ random neighbors of π, where $\#maxNeigb$ is a parameter. Our *MPLS* implementation for *MoQAP* uses the 2-*opt* neighborhood. A neighbor π' of π updates S if neither π nor any solution in S dominates

π'. *MPLS* recursively splits solutions from S into disjoint subsets, which are associated with internal nodes of a recursive tree data structure, to efficiently update S. Each subset stores ideal and nadir points that bound the objective vectors related to the corresponding subtree. The updating procedure discards many branches in the tree by comparing π' to the ideal and nadir points.

3.2 *MO*-MAHM$_E$ implementation

In the *MO*-MAHM$_E$ algorithm, parameters $|N|$ and $\#numPart$ represent the cardinality of the direction vectors set and number of particles, respectively. The method used to compute N, presented by [36], calculates σ (equation (17)), which is a parameter for computing combinations of q non-negative integers $\varphi_1, ..., \varphi_q$ (equation (18)). Equation (19) calculates $\gamma^j = (\gamma_1^j, ..., \gamma_q^j)$, $j = 1, ..., |N|$.

$$|N| = \frac{\prod_{k=1}^{q-1}(\sigma + k)}{(q-1)!} \tag{17}$$

$$\varphi_1 + ... + \varphi_q = \sigma, \quad \varphi_k \in \{1, ..., \sigma\}, k \in \{1, ..., q\} \tag{18}$$

$$\gamma_k^j = \frac{\varphi_k}{\sigma}, \quad k \in \{1, ..., q\} \tag{19}$$

P represents the set with $\#numPart$ particles. The position of the particle is a set of non-dominated solutions. Each $p \in P$ knows its current position and the best position achieved so far, called $p.pbest$. Further, a transition probability matrix $p.\mathbb{P}$ and an indicator matrix $p.Pb$ are associated with p for the learning strategy. A set of ideal points $p.I^1, ..., p.I^{|N|} \in \Re^q$ is associated with p for the velocity operator. $gbest$ is the best position achieved by the swarm, and matrix Gb stores the accumulated improvement on $gbest$ used by the LA scheme.

Algorithm 7 outlines the *MO*-MAHM$_E$ implementation for *MoQAP*. Line 1 computes N. Line 2 initializes set AS of action strategies. For bi-objective instances, *TA*, *NSGA-II*, *MOEA/D*, and *MPLS* are the action strategies; for more than two objectives, *TA*, *A-NSGA-III*, *MOEA/D-DU*, and *MPLS*. Lines 3 and 4 initialize $gbest$ and Gb, respectively. Line 5 creates P such that N, AS, $gbest$, and Gb are global variables available for all particles. For each $p \in P$, the loop between lines 6 and 12 initializes particles' information: current position, $p.pbest$, $p.\mathbb{P}$, $p.Pb$, and $p.I^1, ..., p.I^{|N|}$. The loop between lines 13 and 17 executes particles until satisfying a stop criterion. In this study, the stop criterion is the number of objective function evaluations $\#num\_eval$. Section 3.2.1 details the particle procedure, referred to as $runParticle$. The algorithm outputs $gbest$ at the end (line 18).

Algorithm 7: *MO-MAHM*$_E$ for the *MoQAP*

Input: $|N|$: number of direction vectors
$\#numPart$: number of particles

1 Compute set $N = \{\gamma^1, ..., \gamma^{|N|}\}$
2 $AS \leftarrow$ loadActionStrategies()
3 $gbest \leftarrow \emptyset$
4 $Gb_{(\hat{a},\bar{a})} \leftarrow 0.0, \forall \hat{a}, \bar{a} \in AS$
5 Create P with $\#numPart$ particles
6 **forall the** $p \in P$ **do**
7 $p.current\_position \leftarrow$ random position
8 $p.pbest \leftarrow p.current\_position$
9 $p.\mathbb{P}_{(\hat{a},\bar{a})} = 0.25, \forall \hat{a}, \bar{a} \in AS$
10 $p.Pb_{(\hat{a},\bar{a})} \leftarrow 0.0, \forall \hat{a}, \bar{a} \in AS$
11 $p.I^j \leftarrow \gamma^j, \ \forall j = 1, ..., |N|$
12 **end**
13 **repeat**
14 **forall the** $p \in P$ **do**
15 runParticles(p)
16 **end**
17 **until** satisfying a stop criterion;
18 **return** $gbest$

3.2.1 Particles implementation

All particles implement the learning automata detailed in section 2.4. Learning strategies in related works [13, 29] used the hypervolume unary indicator [58] for assessing the quality of approximation sets in the search. The hypervolume measures the volume in the objective space dominated by an approximation set. This study implemented the hypervolume approximation (HV_{apx}) proposed by [47] because the hypervolume calculation complexity grows with the number of objectives. HV_{apx} assessed the quality of the current position of particles, $pbest$, and $gbest$. Hypervolume requires a reference point to bound the objective space. We used the method addressed by [24] to compute it. This method normalizes each objective in the range [1, 2] and uses (2.1, ..., 2.1) as the reference point.

Particles implement the decision strategy proposed by [29]. After particle p executes an action $a \in AS$ at the $(\tau - 1)$-th step, p selects an action $a' \in AS$ to execute the τ-th step. The selection method can be greedy or based on a roulette-wheel, considering the $p.\mathbb{P}_{(a,a')}(\tau)$ value. In the roulette-wheel method, actions with higher $p.\mathbb{P}_{(a,a')}(\tau)$ values are more likely to be selected. The greedy method selects the action with the highest $p.\mathbb{P}_{(a,a')}(\tau)$. p uses the roulette-wheel during the first $\omega\#num\_eval$ objective function evaluation, where $\omega \in [0, 1]$ is a user-defined parameter. After this phase, the selection can be greedy, with probability ϵ or roulette-wheel probability $1-\epsilon$. Equation (20) computes the ϵ value, where $\#curr\_eval$ is the current number of evaluations.

$$\epsilon = \omega + (1 - \omega)\frac{\#curr\_eval}{\#num\_eval} \tag{20}$$

Algorithm 8: *runParticle* procedure for the *MoQAP*

```
   Input: p : Particle
1  a ← p.decisionStrategy(p.ℙ)
2  new_position ← a.runActionStategy(p.current_position)
3  if HV_apx(new_position) is better than HV_apx(p.pbest) then
4  |   p.pbest ← new_position
5  end
6  if HV_apx(new_position) is better than HV_apx(gbest) then
7  |   gbest ← new_position
8  end
9  p.runLearningStrategy(p.ℙ, p.Pb, Gb)
10 dest ← chooseDestination()
11 p.current_position ←
   velocityOperator(new_position, dest, N, p.I^1, ..., p.I^|N|)
```

Algorithm 8 outlines the *runParticle* procedure. Line 1 selects an action strategy a, and line 2 executes it, such that p's current position is the input for a. *TA*, *NSGA-II*, *A-NSGA-III*, *MOEA/D*, and *MOEA/D-DU* iterate up to $\#maxGen$ generations. *MPLS* executes $\#maxIter$ iterations. $\#maxGen$ and $\#maxIter$ are user-defined parameters. The set output by a is referred to as $new\_position$. If $new\_position$ is better than $p.pbest$ ($gbest$) for the HV_{apx} indicator, then line 4 (line 7) updates $p.pbest$ ($gbest$). Line 9 executes the learning strategy. In line 10, procedure $chooseDestination()$ randomly selects the second set for the velocity operator among $gbest$ and the best position achieved so far by any particle ($p'.pbest$). In the first iteration, the probability of choosing $gbest$ is 1.0. If $new\_position$ is better than $gbest$, the probability of choosing $gbest$ in the $chooseDestination()$ procedure increases by 0.1 (limited to 1.0). Otherwise, it decreases by the same amount (limited to 0.0). The probability of selecting $p'.pbest$ is the complement. In this case, procedure $chooseDestination()$ chooses $p'.pbest$ of any $p' \in P$ with uniform probability. In line 11, the decomposition-based velocity operator moves p from $new\_position$ towards the position selected in the $chooseDestination()$ procedure. Section 3.2.2 details the path generation for the *MoQAP*. The output of the velocity operator becomes the current position of p.

3.2.2 Path generation

The velocity operator uses the path generation implemented by [12]. Let π^t be the targeting solution of the path-relinking trajectory and π an arbitrary solution in the trajectory. The dissimilarity measure between π and π^t is the number of elements such that $\pi_j \neq \pi^t_j$, $j = 1, ..., n$, where π_j and π^t_j represent the location of the j-th element in π and π^t, respectively. Operation $m \in M(\pi, \pi^t)$ generates an intermediate solution $\pi'' = \pi \oplus m$ by swapping two elements in permutation π, say π_i and π_j, $1 \leq i, j \leq n$, such that $\pi_i \neq \pi^t_i$ and $\pi_i = \pi^t_j$. Consequently, $\pi''_j = \pi^t_j$, i.e., π'' is more similar to π^t than π. Equation (21) computes $f_k(\pi'')$, where $\delta(\pi, k, i, j)$ is the difference to $f_k(\pi)$ computed by equation (22) [28].

$$f_k(\pi'') = f_k(\pi) + \delta(\pi, k, i, j), \ \forall k \in \{1, ..., q\} \quad (21)$$

$$\begin{aligned}\delta(\pi, k, i, j) = {} & (\alpha_{jj} - \alpha_{ii})(\beta^k_{\pi_i\pi_i} - \beta^k_{\pi_j\pi_j}) \\ & + (\alpha_{ji} - \alpha_{ij})(\beta^k_{\pi_i\pi_j} - \beta^k_{\pi_j\pi_i}) \\ & + \sum_{o=1, o\neq i,j}^{n} ((\alpha_{oj} - \alpha_{oi})(\beta^k_{\pi_o\pi_i} - \beta^k_{\pi_o\pi_j}) \\ & + (\alpha_{jo} - \alpha_{io})(\beta^k_{\pi_i\pi_o} - \beta^k_{\pi_j\pi_o})) \end{aligned} \quad (22)$$

3.2.3 Particles cooperation

As noted by [13], cooperation among agents can lead to better solutions. In this study, particles cooperate by sharing some information about the search. Variables $gbest$ and Gb are available for the swarm, and any particle can update them. The probability of choosing $gbest$ in the $chooseDestination()$ procedure is available for the swarm. Particle p can read $p'.pbest$ for any $p' \in P$; however, it can update only its $p.pbest$.

4 Computational experiments and results

Computational experiments reported in this section were conducted on cores of Intel Xeon processors E5-2698v3 with 2.3 GHz and 4 GB of RAM per core, running CentOS 6.5, 64 bits, which were provided by the High-Performance Computing Center at UFRN (NPAD/UFRN). Algorithms were implemented with $C++$ and a $GNU\ g++\ 7.1$ compiler.

Section 4.1 describes the test and assessment methodology. Sections 4.2-4.8 report and discuss the results.

4.1 Test and assessment methodology

The first test aimed at investigating the contribution of action strategies to the search. The analysis considered success, failure, transition probabilities, and average HV_{apx} improvement statistics. Success (failure) implies improving (worsening) the current position of the particles for the HV_{apx} indicator. The average HV_{apx} improvement performed by action a represents the total HV_{apx} improvement produced by a divided by the number of a executions.

Further, investigations were related to the benefit of learning strategy, velocity operator, and particle cooperation. Section 4.3 compares results from two *MO*-MAHM$_E$ versions that differ only in the learning strategy: one uses the *LA* scheme, and the other uses the learning strategy that [13] implemented. Section 4.4 compares results from three *MO*-MAHM$_E$ versions that differ only in the velocity operator:

Table 1 Algorithms compared to MO-MAHM$_E$

Hybridization technique	Section 4.6	MO-MAHM HMOEA MOTA	HTA HNSGA NSTA
Hyper- heuristic	Section 4.7	HH-LA	HH-RILA
MoQAP literature	Section 4.8	CoMOLS/D	GRASP

the version referred to as *VO/D* uses the decomposition-based velocity operator, *VO1* uses the velocity operator implemented by [13], and *VO0* is a version without velocity operator. Section 4.5 compares results from parallel and sequential *MO*-MAHM$_E$ versions because particle cooperation is expected to be more robust in parallelism [13]. The parallel version assigns each particle to a virtual thread.

Investigations reported in sections 4.6–4.8 aim to verify whether the results produced by *MO*-MAHM$_E$ are comparable to those obtained by ten algorithms from the literature. Table 1 lists the compared algorithms and classifies them into hybridization techniques, hyper-heuristics, and *MoQAP* literature. The hybridization techniques involve algorithms implemented within the architecture. Table 2 details the hybridization techniques in the *Algorithm* column and the corresponding metaheuristics hybridized for q-objective instances in the *Algorithms Hybridized* column. Hybridizations involved algorithm versions suitable for the number of objectives handled. Hybridizations for the bi-objective instances involve *TA*, *NSGA-II*, *MOEA/D*, and *MPLS*. Those for more than two objectives used *TA*, *A-NSGA-III*, *MOEA/D-DU*, and *MPLS*. *MO-MAHM* is the architecture version proposed by [13]. *HTA* is a memetic algorithm resulting from the hybridization of *MPLS* with *TA*. *HNSGA* for bi-objective instances is a memetic algorithm involving *MPLS* with *NSGA-II*. *HNSGA* hybridizes *MPLS* and *A-NSGA-III* for more than two objectives because *A-NSGA-III* is the *NSGA-II*'s many-objective counterpart. The *HMOEA* definition is analogous concerning *MPLS*, *MOEA/D*, and *MOEA/D-DU*. After applying the evolutionary operators in every iteration, *HTA*, *HMOEA*, and *HNSGA* execute the *MPLS* with the population as its input archive. *NSTA* and *MOTA* are hybrid algorithms proposed by [1], and they combine the transgenetic operators detailed in section 3.1.1 with *NSGA-II* (*A-NSGA-III*) and *MOEA/D* (*MOEA/D-DU*), respectively.

Section 4.7 compares the results produced by *MO*-MAHM$_E$ with those obtained by the *LA*-based hyper-heuristics proposed by [29] (*HH-LA* and *HH-RILA*). For a fair comparison, low-level heuristics for *HH-LA* and *HH-RILA* are algorithms hybridized within the architecture. Finally, section 4.8 compares *MO*-MAHM$_E$ with two algorithms from the *MoQAP* literature: the *GRASP* presented by [28] and the decomposition-based co-evolutionary multi-objective local search (*CoMOLS/D*) proposed by [5]. *GRASP* is included in the comparison because it is a traditional metaheuristic successfully considered for multi-objective optimization, including hybridization [34] and facility-location problems [48]. *CoMOLS/D* is a recent algorithm that employs a decomposition technique similar to that used by *MO*-MAHM$_E$.

Table 2 Summary of hybridization techniques compared to *MO*-MAHM$_E$

Algorithm	q	Algorithms hybridized
MO-MAHM	2	TA, NSGA-II, MOEA/D, MPLS
	> 2	TA, A-NSGA-III, MOEA/D-DU, MPLS
HTA	≥ 2	TA, MPLS
HMOEA	2	MOEA/D, MPLS
	> 2	MOEA/D-DU, MPLS
HNSGA	2	NSGA-II, MPLS
	> 2	A-NSGA-III, MPLS
MOTA	2	TA, MOEA/D
	> 2	TA, MOEA/D-DU
NSTA	2	TA, NSGA-II
	> 2	TA, A-NSGA-III

4.1.1 Instances

The benchmark set contains 27 *MoQAP* symmetric instances built with the generator that [22] presented. There are 15 real-like (KRL) instances and 12 uniformly random ones ($KUNI$). The triple (n, q, ID) identifies an instance, where $n \in \{20, 30, 40, 50\}$ represents the number of locations/facilities, $q \in \{2, 3, 4\}$ represents the number of objectives, and $ID \in \{1, 2, 3, 4, 5\}$.

KRL distance matrix entries are the Euclidean distance between pairs of points in the plane. Expression (23) computes KRL flow matrices entries, where $\mu_1 \in \mathbb{Z}$, $\mu_2 \in \mathbb{Z}^+$, $\mu_1 < \mu_2$, and χ represents a random value uniformly distributed in $[0, 1)$. A fraction of entries in the k-th flow matrix, $1 < k \leq q$, is correlated with the corresponding entry in the first KRL flow matrix. An *overlap* parameter (ov) controls this fraction, and $cf_k \in [-1, 1]$ is the correlation factor concerning the k-th flow matrix. Table 3 presents the parameters for the *KRL* instances.

$$\lfloor 10^{((\mu_2-\mu_1)*\chi+\mu_1)} \rfloor \tag{23}$$

Table 3 Parameters for the *KRL* instances

ID	1	2	3	4	5	1	2	3	1	2	3	1	1	2	3
n	20	20	20	20	20	30	30	30	40	40	40	50	50	50	50
q	2	2	2	2	2	3	3	3	3	3	3	3	4	4	4
cf_2	0.0	0.4	−0.4	0.4	0.4	0.4	0.7	−0.4	0.4	0.7	−0.4	0.4	−0.6	0.6	0.8
cf_3	–	–	–	–	–	0.0	−0.5	−0.4	0.0	−0.5	−0.4	−0.8	0.4	0.0	−0.2
cf_4	–	–	–	–	–	–	–	–	–	–	–	–	-0.2	0.0	0.7
ov	1.0	1.0	1.0	1.0	0.5	0.7	0.7	0.2	0.7	0.7	0.2	0.7	0.7	0.7	0.7
μ_1	−2	–	−2	−5	−5	−2	−2	−2	−2	−2	−2	−2	−2	−2	−2
μ_2	4	4	4	5	5	4	4	4	4	4	4	4	4	4	4

Table 4 Parameters for the *KUNI* instances

ID	1	2	3	1	2	3	1	2	3	1	2	3
n	20	20	20	30	30	30	40	40	40	50	50	50
q	2	2	2	3	3	3	3	3	3	4	4	4
cf	0.0	0.7	–0.7	0.0	0.4	–0.4	0.0	0.6	–0.6	0.0	0.8	–0.8

Table 5 Parameter settings for decision and learning strategies

Procedure	Parameter	Procedure	Parameter
Decision strategy	$\omega = 0.44$	*Learning strategy*	$\nu_{pb} = 0.3$ $\nu_{gb} = 0.7$

The distance and flow matrices entries for the $KUNI$ instances are random integers uniformly distributed in the range [1, 100]. A parameter $cf \in [-1, 1]$ correlates entries in the first $KUNI$ flow matrix with the corresponding entry in the other flow matrices. Table 4 presents the parameters for $KUNI$ instances.

4.1.2 Quality indicators and statistical tests

Results for each instance concern 30 independent executions of each algorithm. Hypervolume (HV) [58] and inverted generational distance (IGD) [7] assess the quality of the approximation sets produced by algorithms. For each instance, calculating these indicators requires a reference set R. We computed R by filtering the non-dominated vectors from the union of all approximation sets produced by all algorithms tested. The HV results consist of the difference between the approximation set produced by the algorithm and R. As suggested by [24], we normalized objective vectors in the range [1, 2] and used (2.1, ..., 2.1) as the reference point for HV. IGD presents the diversity among the objective vectors in an approximation set $Y^{*\prime}$. Equation (24) computes IGD, where $ds(r, Y^{*\prime})$ denotes the distance between $r \in R$ and the nearest objective vector in $Y^{*\prime}$. The lower the HV and IGD values, the better the results. Appendix A presents the quality indicator values.

$$IGD(Y^{*\prime}, R) = \frac{\sum_{r \in R} ds(r, Y^{*\prime})}{|R|} \tag{24}$$

Statistical tests verify significant differences among results. For experiments involving two algorithms (sections 4.3, 4.5), the Mann–Whitney test [8] performs a one-tailed comparison. For more than two algorithms (sections 4.2, 4.4, 4.6, 4.7, 4.8), the Kruskal–Wallis test [8] applies two comparison phases. The first phase checks statistical differences among multiple independent results from the algorithms. If the first phase indicates significant differences, the second one performs one-tailed pairwise comparisons between the results provided by MO-MAHM$_E$ and those from the other algorithms. Both the Mann–Whitney and Kruskal–Wallis tests considered a significance level of 0.05 and were provided by the PISA framework [3].

4.1.3 Parameters setting

The irace package [31] tuned the parameters for all algorithms. MO-MAHM$_E$ used $\#numPart = 4$ particles. The decomposition-based velocity operator used $|N| = 55$ direction vectors. Table 5 presents the parameter settings for the decision and learning strategies.

Table 6 exhibits the parameters of the action strategies. Except for $\#maxGen$, Table 6 exhibits the values of the parameters of the hybrid algorithms presented in Table 2. *HH-LA* and *HH-RILA* used parameters presented in Table 5 for the decision strategy. The multiplier required by *HH-LA* and *HH-RILA* for the learning strategy was 0.75. *GRASP* and *CoMOLS/D* used parameters defined by [28] and [5], respectively. The stopping criterion was 10^6 evaluations of the objective function for MO-MAHM$_E$ and compared algo-

Table 6 Parameter settings for action strategies

Algorithm	Parameter	Algorithm	Parameter	Algorithm	Parameter
TA	#maxGen = 9 #popSize = 55 #probPlasm = 0.95 #probFactor = 0.1 #intGerSet = 3 #sizeRepo = 5	*NSGA-II*	#popSize = 55 #maxGen = 10 #rateCross = 0.92 #rateMut = 0.10	*MOEA/D*	#popSize = 55 #maxGen = 9 #rateCross = 0.96 #rateMut = 0.09 #neigb = 20
MPLS	#maxIter = 10 #maxNeigb = 10	*A-NSGA-III*	#popSize = 55 #maxGen = 10 #rateCross = 0.91 #rateMut = 0.02	*MOEA/D-DU*	#popSize = 55 #maxGen = 9 #rateCross = 0.91 #rateMut = 0.08 #neigb = 21 #probNeigb = 0.34 #maxReplc = 41

rithms in Table 1. The number of evaluations was distributed equally among the particles. We used the adaptive grid archiving technique proposed by [23], limited to 300 solutions, for implementing archives of non-dominated solutions required by the algorithms.

4.2 Impact of action strategies

All action strategies presented non-zero rates of successes, failures, and average HV_{apx} improvement for all instances. Except for the 20.2 2-objective *KUNI* instance, the actions presented, on average, a success rate greater than failure. The 20.2 2-objective *KUNI* instance is small-sized and highly correlated (0.7). These features related it to a small Pareto-optimal set and made it an easy instance. Therefore, the action strategies tend to find small approximation sets on such an instance.

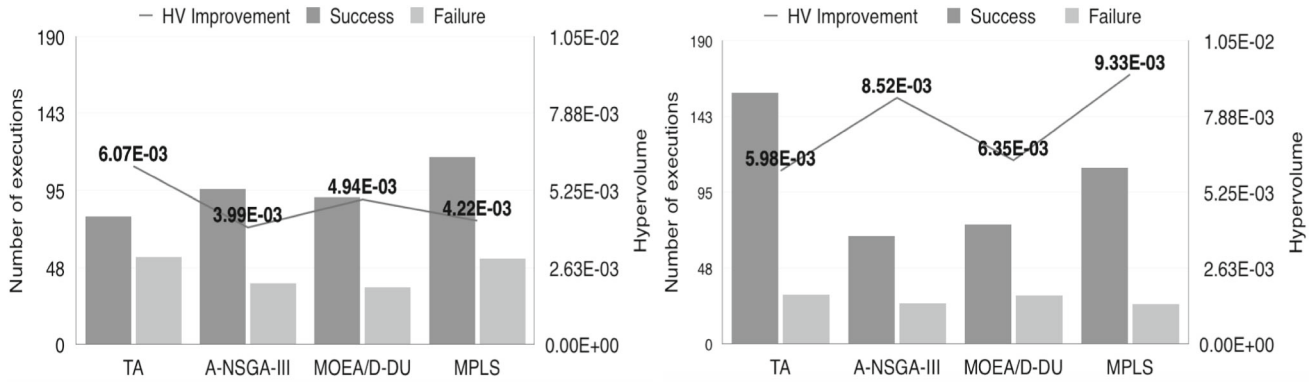
These results confirm that all heuristics hybridized within the *MO*-MAHM$_E$ contribute to the search on all instances. Those results were similar among the instances; Figure 1 illustrates them for 50.1 4-objective *KRL* and 40.1 3-objective *KUNI*. These instances were selected only for illustration purposes as they are representative of each group. The x-axis shows the action strategies. The figure has two y-axes. The first (left) one is related to the bars and shows the average number of executions in which an action strategy succeeded or failed in improving the current position of the particles. The second (right) y-axis is related to the line and shows the average HV_{apx} improvement. In both instances, action strategies presented a success rate greater than failure and non-zero HV_{apx} improvement.

The Kruskal–Wallis test checked statistical differences among the action strategies related to success rates and average HV_{apx} improvement. Figures 2–5 show a 4×4 matrix for each instance (n, q, ID), whose main diagonal is marked by the $\cdot$ symbol to summarize these results. Results for the success rates and average HV_{apx} improvement appear below and above the main diagonal, respectively. The $>$ ($<$) symbol indicates that the action strategy in the row was conclusively more (less) successful or produced better (worse) average HV_{apx} improvement than the one in the corresponding column. The $\sim$ symbol represents inconclusive results.

The first Kruskal–Wallis test did not indicate significant differences among the action strategies for the HV_{apx} improvement on 4 from 15 (26.67%) *KRL* instances and 8 from 12 (66.67%) *KUNI* instances. In Figures 2-5, such instances correspond to matrices where the $\sim$ symbol appears in all entries above the main diagonal. For 2-objective instances where the first Kruskal–Wallis has passed, the pairwise test showed that *TA* presented similar or worse improvements than those of the other algorithms (Figures 2 and 4). For 3- and 4-objective instances, *TA* presented similar or better improvement (Figures 3 and 5). The only exception was the (40,3,2) *KUNI* instance, on which *MOEA/D-DU* presented the best HV_{apx} improvement. Pairwise comparisons for the HV_{apx} improvement among *MPLS*, *NSGA-II*, and *MOEA/D* (*A-NSGA-III* and *MOEA/D-DU* for 3- and 4-objective) were inconclusive for most cases. These results suggest a tendency of *TA* to produce significantly good solutions in hybridizations for more than two objective functions.

The first Kruskal–Wallis test indicated significant differences regarding the success rates on all instances. The pairwise test indicated that *TA* was the most successful on 2-objective instances from *KRL* and *KUNI* sets (Figures 2 and 4, respectively), followed by *MPLS*. Similar results occurred for 3- and 4-objective *KUNI* instances (Figure 5), except on (30,3,3), (40,3,3), and (50,4,3) instances, where *MPLS* was the most successful. Further, *MPLS* was successful on most 3- and 4-objective *KRL* instances (Figure 3).

In most instances, the average number of successes is not directly correlated with the average HV_{apx} improvement, i.e., an action strategy can be more successful conclusively



(a) 50.1 4-objective *KRL* instance (b) 40.1 3-objective *KUNI* instance

Fig. 1 Average number of successes, failures, and average hypervolume improvement

Fig. 2 Kruskal-Wallis results regarding the number of successes and the average HV improvement for 2-objective *KRL* instances

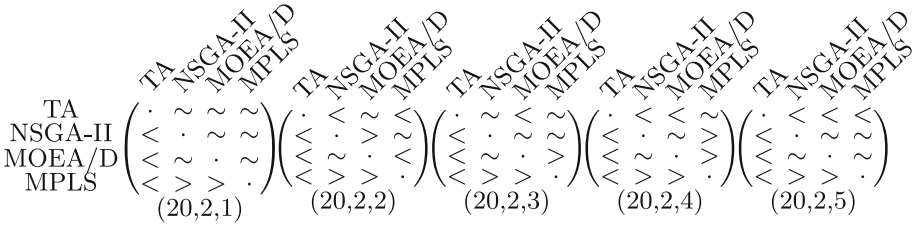
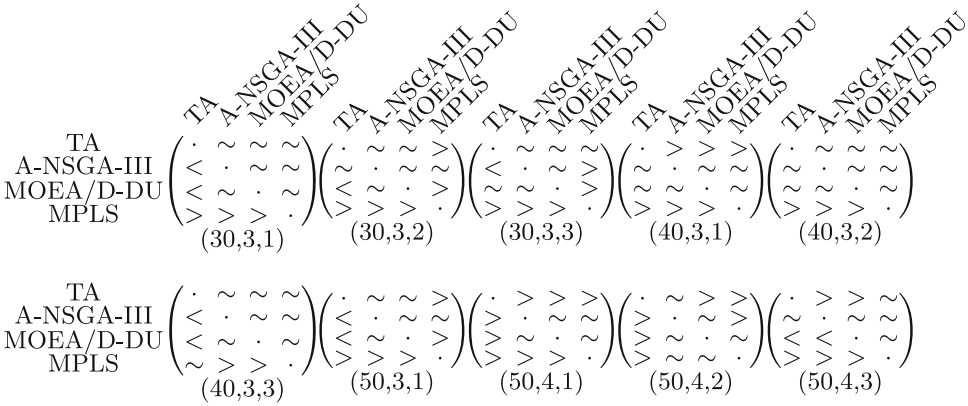


Fig. 3 Kruskal-Wallis results regarding the number of successes and the average HV improvement for 3 and 4-objective *KRL* instances.



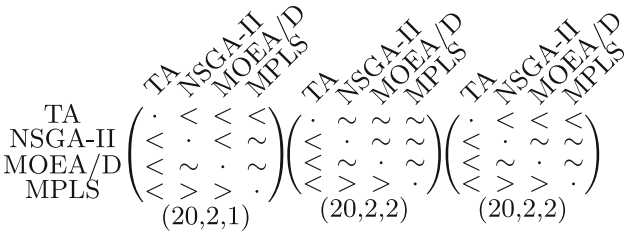


Fig. 4 Kruskal-Wallis results regarding the number of successes and the average HV improvement for 2-objective *KUNI* instances

but present worse or similar average HV_{apx} improvement than another. For example, in the (30, 3, 2) and (50, 3, 1) *KRL* instances (Figure 3), *MPLS* was more successful but presented worse average HV_{apx} improvement than *TA* and *MOEA/D-DU*; in the (50, 4, 1) *KRL* (Figure 3) and (30, 3, 3) *KUNI* (Figure 5), *MOEA/D-DU* was more successful than *TA*, and the latter produced better HV_{apx} than the former. This is attributed to the variation of HV_{apx} improvement that the action strategies produce during the *MO*-MAHM$_E$ execution. An action strategy achieves a significant HV_{apx} contribution at initial iterations and produces decreasing ones in subsequent iterations. Although the initial contribution leads particles to prefer such an action at this search point, subsequent failures (or non-significant contributions) lead the particles to execute successful actions in future iterations.

Figures 6 and 7 present the transition matrices computed by the *LA* scheme for *KRL* and *KUNI* instances, respectively. Matrices show the average transition probability, written as a percentage, for switching from the action strategy that names

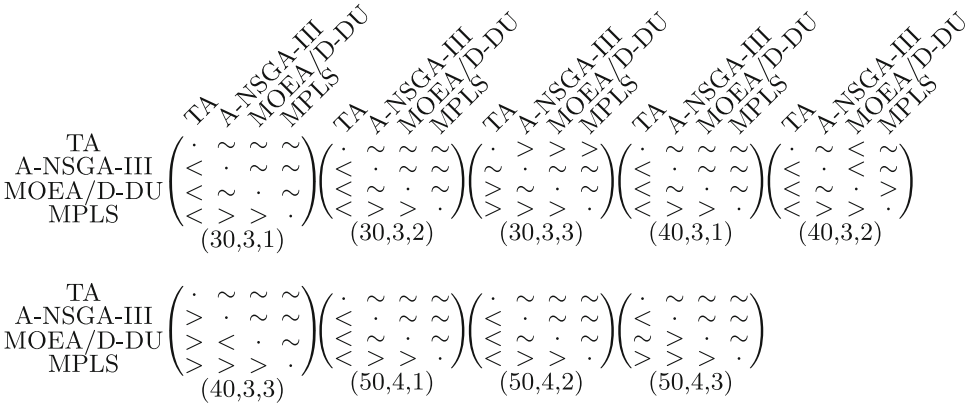
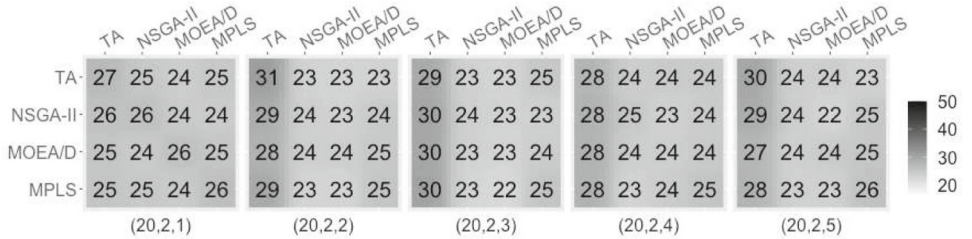
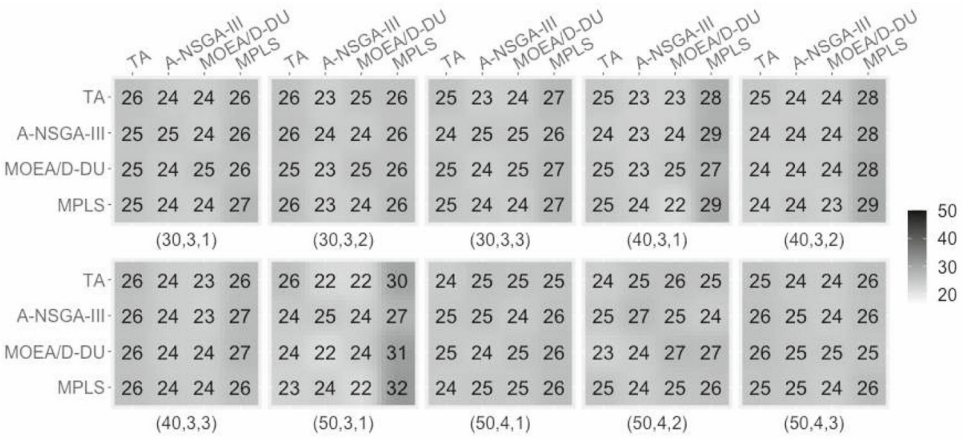


Fig. 5 Kruskal-Wallis results regarding the number of successes and the average HV improvement for 3 and 4-objective *KUNI* instances



(a) 2-objective *KRL*



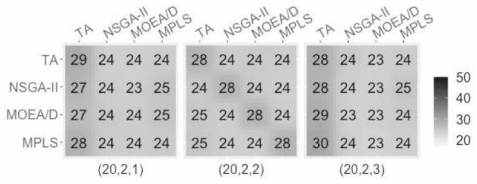
(b) 3 and 4-objective *KRL*

Fig. 6 Average transition probability matrices (written as a percentage) produced by the *LA* scheme for *KRL* instances

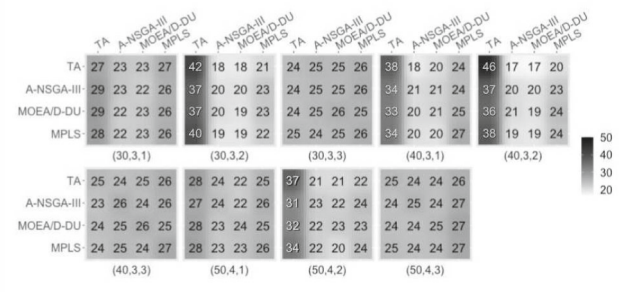
the row to that in the corresponding column. The darker the color, the higher the transition probability. The figures indicate that particles preferred successful action strategies. For example, particles preferred *TA* for the *KUNI* set (Figure 7) and 2-objective *KRL* instances (Figure 6). Similar results occurred for the *MPLS* for the (40, 3, 1), (40, 3, 2), and (50, 3, 1) *KRL* instances. It occurred because the *LA* scheme assigns less weight to past HV_{apx} contributions. Unsuccessful action strategies (even if they have achieved significant HV_{apx} contributions in past iterations) decrease their reward rates over time. Actions that contribute more frequently tend to increase or maintain their reward rates; therefore, successful action strategies are more likely to execute.

4.3 Experiments concerning the learning strategy

Besides the 20.3 2-objective *KRL* instance, all p-values from the Mann–Whitney one-tailed test were less than 0.05. The *LA*-based *MO*-MAHM$_E$ version conclusively produced better *HV* and *IGD* values than the version that used the learning strategy implemented by [13]. For the 20.3 2-objective *KRL* instance, the statistical test did not indicate significant differences (p-value = 0.055) for *HV*; however, it was conclusive in favor of the *LA*-based *MO*-MAHM$_E$ version for the *IGD* (p-value = 0.002). The boxplots in Figure 8 illustrate the distributions of the *HV* and *IGD* results from the two algorithms. The results for the 20.3 2-objective *KRL* instance can be attributed to the small size and number of objectives.



(a) 2-objective *KUNI*



(b) 3 and 4-objective *KUNI*

Fig. 7 Average transition probability matrices (written as a percentage) produced by the *LA* scheme for *KUNI* instances

Those results suggest that the *LA* scheme is beneficial for the search. *LA* structural features may explain such benefits. The transition matrix allows the agents to consider the action executed in the previous step to choose one in the current step. *LA* adequately updated the transition probabilities and prevented any action strategy from being "forgotten" by the swarm.

4.4 Experiments concerning the velocity operator

The first Kruskal–Wallis test indicated significant differences among the *VO/D*, *VO1*, and *VO0* for all instances for the HV. The same occurred for most instances with IGD. Exceptions concerning the IGD are the 20.1 and 20.2 2-objective *KRL* instances and the 20.2 2-objective *KUNI* instance. These results can be a consequence of the small size (20 locations/facilities) and number of objectives, which do not pose sufficient difficulties in the algorithms. In addition, instances generated with increasing correlation factors tend to be easier because they are related to small Pareto-optimal sets. The correlation factors that generated these instances (0.0, 0.4, and 0.7, respectively) may have influenced such results.

Besides the *IGD* values on the 20.2 2-objective *KUNI* instance, for which the first statistical test was not passed, the pairwise test obtained p-values < 0.05 for all 2-objective *KUNI* instances related to both *HV* and *IGD*. These results conclusively indicate that *VO/D* exhibited the best performance for 2-objective *KUNI* instances.

Table 7 lists p-values from the pairwise test on 2-objective *KRL* instances. P-values < 0.05 indicate the best results for the *VO/D*; p-values > 0.95 indicate the best results for the version in the column; p-values between 0.05 and 0.95 indicate inconclusive results. The $H0$ symbol indicates that the first Kruskal–Wallis test did not indicate significant differences. Table 7 shows that *VO/D* is competitive on most 2-objective *KRL* instances, mainly against *VO0*. *VO1* is conclusively better only on the 20.1 instance for HV. There are two inconclusive HV results and they are related to the comparison with *VO1* on 20.2 and 20.5 instances. As stated earlier, the small size, small number of objectives, and correlation factor 0.4 influenced the results on those instances.

The pairwise test showed that *VO/D* behaved conclusively better (p-values$<$ 0.05) than *VO1* on all 3- and 4-objective instances from both sets and for both quality indicators. Besides the 50.2 4-objective *KUNI* instance, the same occurred for the comparison with *VO0*. For the 50.2 4-objective *KUNI* instance, the pairwise test indicated a statistical similarity between *VO/D* and *VO0* for *IGD* results (p-value $= 0.082$), and indicated that *VO/D* produced the best *HV* (p-value = 0.035) conclusively. The reason is the high correlation factor (0.8) used to generate such an instance. Figure 9 illustrates the distribution of *HV* and *IGD* values for the 50.2 4-objective *KUNI* instance.

Results reported in this section confirmed that the decomposition-based velocity operator benefits the search process. The benefit lies in moving the swarm towards high-quality and diverse solutions, which causes particles to escape from local optima. In addition, the advantage increases with the number of objectives and for small correlation factors. This fact indicates that the decomposition approach can adequately deal with complex Pareto-fronts and exponentially sized Pareto-optimal sets.

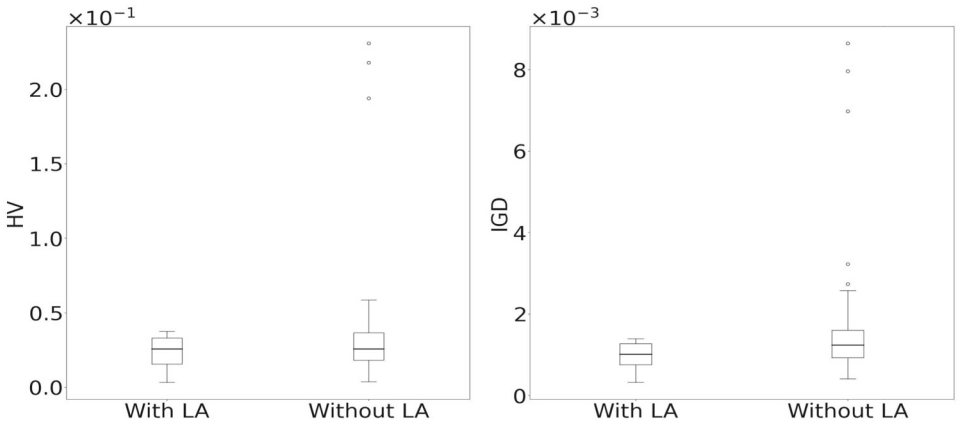
4.5 Experiments concerning particle cooperation

Tables 8 and 9 exhibit p-values from the Mann–Whitney one-tailed test for *KRL* and *KUNI* instances, respectively. P-values < 0.05 are conclusive in favor of the parallel version; those between 0.05 and 0.95 are inconclusive. There are no conclusive p-values in favor of the sequential version.

The parallel version performed conclusively better than the sequential one on most *KRL* instances with three and four objectives. P-values were inconclusive on most 2-objective *KRL* instances and the 40.2 3-objective regarding the IGD. From 2-objective *KRL* instances, HV and IGD conclusively agree in favor of the parallel version only on the 20.4 instance.

The statistical test showed that the parallel version conclusively produced the best HV values on 8 from 12 *KUNI* instances (66.67%), and the best IGD values on 5 from 12 (41.66%). The remaining p-values are inconclusive.

Results presented in this section reinforce that cooperation tends to benefit the search. As [13] explained, cooperation is stronger in parallelism because individual agents more promptly access the information shared by the swarm. In con-

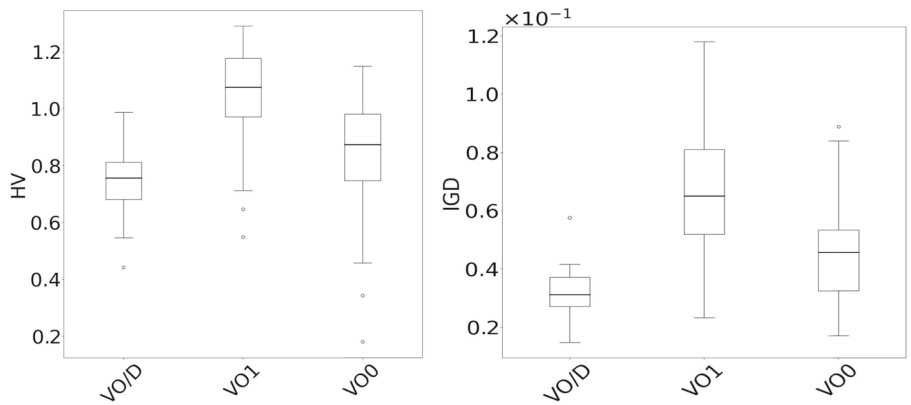


(a) *HV* (b) *IGD*

Fig. 8 Boxplots concerning *HV* and *IGD* values for 20.3 2-objective *KRL* instance

Table 7 Kruskal-Wallis pairwise *p*-values regarding the impact of velocity operator on 2-objective *KRL* instances

	VO1		VO0	
n.ID	HV	IGD	HV	IGD
20.1	0.992	H0	0.016	H0
20.2	0.338	H0	0.005	H0
20.3	0.034	0.002	0.000	0.000
20.4	0.048	0.000	0.000	0.000
20.5	0.149	0.001	0.000	0.000



(a) *HV* (b) *IGD*

Fig. 9 Boxplots concerning *HV* and *IGD* values for 50.2 4-objective *KUNI* instance

Table 8 Mann-Whitney *p*-values concerning the parallelism on *KRL* instances

q	2					3							4		
n.ID	**20.1**	**20.2**	**20.3**	**20.4**	**20.5**	**30.1**	**30.2**	**30.3**	**40.1**	**40.2**	**40.3**	**50.1**	**50.1**	**50.2**	**50.3**
HV	0.104	0.230	0.599	0.004	0.121	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
IGD	0.068	0.500	0.225	0.000	0.110	0.000	0.021	0.000	0.000	0.094	0.001	0.002	0.019	0.009	0.004

Table 9 Mann-Whitney *p*-values concerning the parallelism on *KUNI* instances

q	2			3						4		
n.ID	**20.1**	**20.2**	**20.3**	**30.1**	**30.2**	**30.3**	**40.1**	**40.2**	**40.3**	**50.1**	**50.2**	**50.3**
HV	0.074	0.315	0.000	0.000	0.002	0.000	0.015	0.356	0.000	0.000	0.407	0.000
IGD	0.253	0.782	0.001	0.000	0.000	0.000	0.076	0.356	0.021	0.225	0.323	0.389

trast, agents tend to starve such information in the sequential version.

4.6 Comparison with hybrid versions of the algorithms implemented within the *MO*-MAHM$_E$

The first Kruskal–Wallis test indicated significant differences for all comparisons. The pairwise test showed that *MO*-MAHM$_E$ produced conclusively better HV and IGD results (p-values < 0.05) than those of *MO-MAHM*, *HMOEA*, and *HNSGA* for all *KRL* and *KUNI* instances. These results confirm the hypotheses that *MO*-MAHM$_E$ performs better than *MO-MAHM*, and therefore, it is more suitable for 2-, 3- and 4-objective instances. In addition, these results confirm that the proposed architecture finds better and more diverse solutions than the state-of-the-art many-objective metaheuristics hybridized with local search (*HNSGA* and *HMOEA*).

The *MO*-MAHM$_E$ algorithm yielded significantly superior HV and IGD results (p-values < 0.05) compared to *MOTA* and *NSTA* for all *KRL* instances. Table 10 lists the p-values for comparisons with *HTA* on *KRL* instances, indicating that most p-values favor *MO*-MAHM$_E$ (p-values$<$ 0.05) in terms of both quality indicators. *HTA* only outperformed *MO*-MAHM$_E$ for HV and IGD results (p-values > 0.95) on the 20.1 2-objective instance. There are five instances with inconclusive p-values: one for HV (20.2 2-objective) and four for IGD (30.3 3-objective and three 50-sized 4-objective).

Figure 10 illustrates the HV and IGD results for the 50.1 4-objective *KRL* instance. The boxplots reveal that, in this instance, *MO*-MAHM$_E$ achieved better *HV* values than the other algorithms. The same occurred for *IGD*, except for the similarity between *MO*-MAHM$_E$ and *HTA*. Despite no statistical differences, Table 19 in Appendix A shows that *MO*-MAHM$_E$ produced better mean IGD results than *HTA* for the 50.1 4-objective *KRL* instance.

Table 11 shows the pairwise p-values comparing *MO*-MAHM$_E$ with *HTA*, *MOTA*, and *NSTA* on *KUNI* instances. *MO*-MAHM$_E$ produced conclusively better HV results than *HTA*, *MOTA*, and *NSTA* on 7 (58.33%), 11 (91.67%), and 10 (83.33%) from 12 instances, respectively. *MO*-MAHM$_E$ yielded conclusively better IGD results on 3 (25.00%), 6 (50.00%), and 8 (66.67%) instances, respectively. Therefore, *MO*-MAHM$_E$ produced the best *HV* values for most *KUNI* instances; however, the same did not occur for the *IGD*. The results presented in Table 11 stem from the configurations of the *KUNI* set. Unlike *KRL* instances, *KUNI* instances consist of uniformly random values generated with a single correlation factor parameter. With an increase in the correlation factors, *KUNI* instances tend to become easier, making *HTA*, *MOTA*, and *NSTA* more competitive. For example, these hybrid algorithms conclusively produced better and more diverse solutions in the 50.2 4-objective *KUNI* instance (correlation factor 0.8). Conversely, *MO*-MAHM$_E$ achieved the best HV and IGD results in the 20.3 2-objective, 40.3 3-objective, and 50.3 4-objective instances (with correlation factors of -0.7, -0.6, and -0.8, respectively). The only exception is the 30.3 3-objective instance (correlation factor -0.4), where *HTA* and *MOTA* yielded better IGD values than those of *MO*-MAHM$_E$.

The *HTA* has been the second most competitive algorithm for *KUNI* instances because *TA* and *MPLS* were action strategies that presented the best improvements and success rates. Further, we observed that transgenetic operators led many-objective optimizers tested in this study (*A-NSGA-III* and *MOEA/D-DU*) to be competitive on *KUNI* instances. This is the case, for example, for the 40.2 3-objective *KUNI* instance, as illustrated in Figure 11.

4.7 Comparison with hyper-heuristics

The first Kruskal–Wallis test identified significant differences among *MO*-MAHM$_E$, *HH-LA*, and *HH-RILA* for all instances and both quality indicators. The pairwise test indicated that *MO*-MAHM$_E$ consistently achieved the best HV and IGD results for all instances, including those with three and four objectives. Appendix A details *HV* and *IGD* results.

Although *MO*-MAHM$_E$, *HH-LA*, and *HH-RILA* use the same learning, decision, and action strategies, *MO*-MAHM$_E$ takes advantage of concepts from *PSO* and multi-agent paradigms. For example, the multi-agent paradigm enabled the simultaneous exploration of different regions of the search space. *MO*-MAHM$_E$ can simultaneously explore different solutions because each agent (particle) maintains its current set of non-dominated solutions (current position) and the best one achieved so far ($pbest$). The velocity operator, another fundamental *PSO* concept, significantly benefited the results.

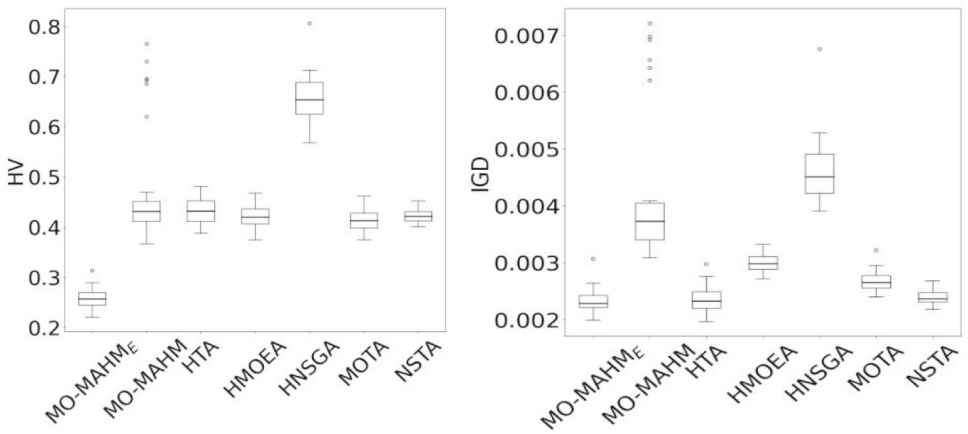
4.8 Comparison with algorithms from the *MoQAP* literature

The first Kruskal–Wallis test indicated significant differences regarding both indicators for all instances. According to the pairwise test, *MO*-MAHM$_E$ produced conclusively better HV and IGD results (p-values$<$ 0.05) than *GRASP* on most instances from both *KRL* and *KUNI* sets. The exception was the 50.3 4-objective *KUNI* instance, for which the pairwise test was inconclusive (p-value$=$ 0.629) regarding the HV.

Table 12 exhibits the pairwise p-values between *MO*-MAHM$_E$ and *CoMOLS/D* for *KUNI* instances. *MO*-MAHM$_E$ produced conclusively better HV and IGD results (p-values$<$ 0.05) than *CoMOLS/D* for most *KUNI* instances. *CoMOLS/D* produced conclusively better results (p-value$>$ 0.95) for only two *KUNI* instances: 50.1 4-objective for both HV and IGD, and 50.3 4-objective for HV.

Table 10 Kruskal-Wallis pairwise p-values between *MO*-MAHM$_E$ and *HTA* on *KRL* instances

q	2					3							4		
n.ID	**20.1**	**20.2**	**20.3**	**20.4**	**20.5**	**30.1**	**30.2**	**30.3**	**40.1**	**40.2**	**40.3**	**50.1**	**50.1**	**50.2**	**50.3**
HV	1.000	0.088	0.000	0.000	0.001	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
IGD	0.989	0.000	0.000	0.000	0.001	0.000	0.000	0.862	0.000	0.000	0.000	0.000	0.183	0.349	0.919



(a) HV (b) IGD

Fig. 10 Boxplots concerning *HV* and *IGD* values for 50.1 4-objective *KRL* instance.

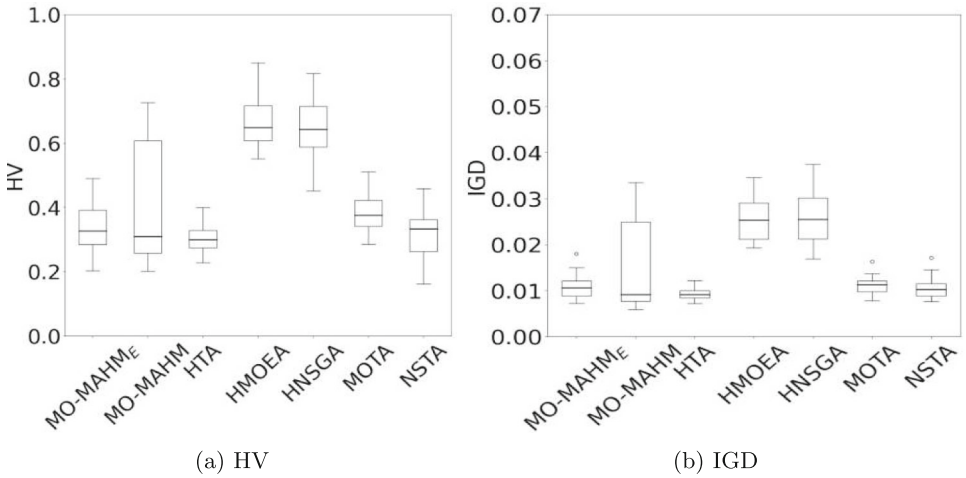
Table 11 Kruskal-Wallis pairwise p-values between *MO*-MAHM$_E$ and HTA, MOTA and NSTA on *KUNI* instances

		HTA		MOTA		NSTA	
q	**n.ID**	**HV**	**IGD**	**HV**	**IGD**	**HV**	**IGD**
2	**20.1**	0.999	0.988	0.000	0.071	0.005	0.114
	20.2	0.347	0.982	0.000	0.000	0.000	0.057
	20.3	0.000	0.000	0.000	0.000	0.000	0.000
3	**30.1**	0.000	0.990	0.000	0.303	0.000	0.000
	30.2	0.982	1.000	0.000	0.009	0.000	0.000
	30.3	0.000	1.000	0.000	1.000	0.000	0.000
	40.1	0.000	1.000	0.000	0.853	0.000	0.000
	40.2	0.996	0.999	0.002	0.069	0.878	0.507
	40.3	0.000	0.000	0.000	0.000	0.000	0.000
4	**50.1**	0.000	1.000	0.024	0.041	0.000	0.000
	50.2	1.000	1.000	0.991	0.996	1.000	1.000
	50.3	0.000	0.000	0.000	0.000	0.000	0.000

Table 13 shows the results for *KRL* instances. For the HV indicator, *MO*-MAHM$_E$ produced the best results (p-value<0.05) on 8 from 15 instances (53.33%), *CoMOLS/D* (p-value>0.95) on 5 (33.33%), and two results were inconclusive (13.33%). *MO*-MAHM$_E$ produced the best IGD on 12 from 15 (80.00%), *CoMOLS/D* on 2 (13.33%), and the comparison was inconclusive on one instance (6.67%). Figure 12 illustrates the results for the 40.1 3-objective *KRL* instance, wherein *MO*-MAHM$_E$ and *HTA* produced similar *HV* results, while *MO*-MAHM$_E$ achieved the best *IGD* distribution.

5 Conclusion and future work

This paper proposed a multi-agent architecture *MO*-MAHM$_E$ to design hybrid metaheuristics for solving problems with two or more objective functions. *MO-MAHM*, an architecture previously proposed for bi-objective problems, was extended in this study. The similarity between *MO-MAHM* and *MO*-MAHM$_E$ lies in the inspiration from the PSO framework and multi-agent paradigm. They differ in the velocity operator and the learning strategy. The novel architecture employs a decomposition-based velocity operator to ensure an appropriate balance between diversification and intensification,



(a) HV (b) IGD

Fig. 11 Boxplots concerning *HV* and *IGD* values for 40.2 3-objective *KUNI* instance.

Table 12 Kruskal-Wallis pairwise *p*-values between *MO*-MAHM$_E$ and *CoMOLS/D* on *KUNI* instances

q	2			3						4		
n.ID	**20.1**	**20.2**	**20.3**	**30.1**	**30.2**	**30.3**	**40.1**	**40.2**	**40.3**	**50.1**	**50.2**	**50.3**
HV	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.003	1.000	0.000	1.000
IGD	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.000	0.000	0.000

Table 13 Kruskal-Wallis pairwise *p*-values between *MO*-MAHM$_E$ and *CoMOLS/D* on *KRL* instances

q	2					3							4		
n.ID	**20.1**	**20.2**	**20.3**	**20.4**	**20.5**	**30.1**	**30.2**	**30.3**	**40.1**	**40.2**	**40.3**	**50.1**	**50.1**	**50.2**	**50.3**
HV	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.999	0.400	0.446	0.974	0.000	1.000	1.000	1.000
IGD	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.001	0.000	0.772	0.000	0.000	0.000	0.998	1.000

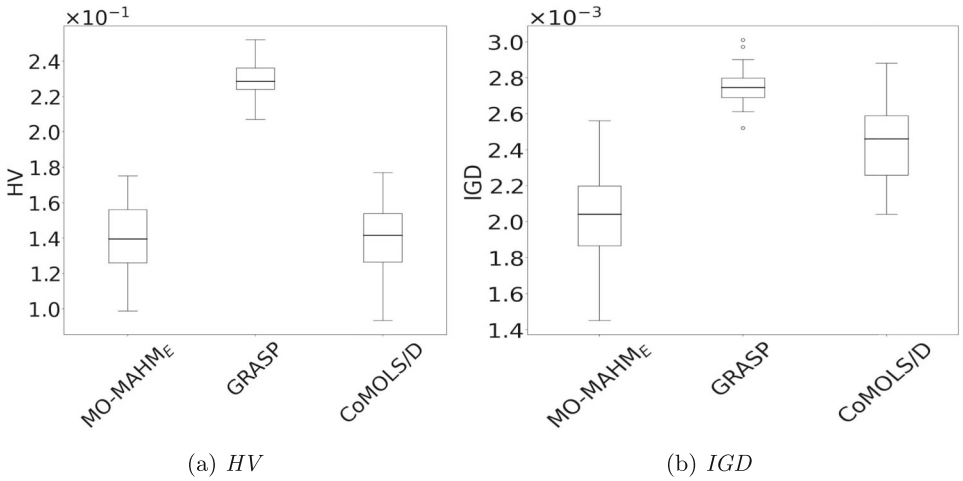
and reinforcement learning automata to support the decisions of the particles.

We presented an application for *MoQAP* that hybridized two many-objective evolutionary algorithms (*A-NSGA-III* and *MOEA/D-DU*), a transgenetic algorithm from the *MoQAP* literature, and a many-objective Pareto local search to investigate the *MO*-MAHM$_E$ performance. The experimental analyses revealed that these algorithms contributed significantly to the hybridization, i.e., *MO*-MAHM$_E$ does well in combining and exploiting the potentiality of individual algorithms.

This study revealed that *MO*-MAHM$_E$ successfully employed decomposition and reinforcement learning methods for tackling problems with two or more objective functions. The statistical test results indicated that the velocity operator and learning automata benefited the search process for more than two objectives and instances related to exponentially sized Pareto-optimal sets. The experiments included instances with up to four objectives, and the analyses determined the quality of the approximation sets (measured by the hypervolume) and solutions diversity (measured by inverted generational distance).

We compared *MO*-MAHM$_E$ with ten algorithms. *MO*-MAHM$_E$ produced conclusively better results than *MO-MAHM* and many-objective memetic algorithms for most instances. *MO*-MAHM$_E$ performed better than two *LA*-based hyper-heuristics and two *MoQAP* algorithms. These results suggest that the proposed architecture is competitive in dealing with more than two objectives. Furthermore, the implementation presented in this paper is a novel *MoQAP* state-of-the-art algorithm (Tables 14, 15, 16, 17, 18 and 19).

Future work will include the investigation of some remaining open topics. For example, we intend to investigate the *MO*-MAHM$_E$ performance for an increasing number (more than four) of objective functions. Further, *MO*-MAHM$_E$ will be enhanced using other machine learning techniques, such as armed bandit models and meta-learning. Finally, we also intend to investigate the benefit of employing simultaneously different learning strategies.



(a) *HV* (b) *IGD*

Fig. 12 Boxplots concerning *HV* and *IGD* values for 40.1 3-objective *KRL* instance

A Supplementary Results

This appendix presents mean results concerning *HV* and *IGD* quality indicators for comparisons in sections 4.3-4.8. In the following tables, bold entries are the lowest value among the algorithms in the corresponding row.

Table 14 Mean *HV* results for learning strategy, velocity operator, and cooperation

q	n.ID	MO- MAHM$_E$	Without LA	With VO1	Without Velocity Operator	Without Cooperation
KRL						
2	**20.1**	5.92E-02	9.69E-02	**4.83E-02**	7.74E-02	6.12E-02
	20.2	3.63E-02	9.16E-02	**3.51E-02**	4.32E-02	3.96E-02
	20.3	**2.25E-02**	4.44E-02	2.95E-02	4.28E-02	2.28E-02
	20.4	**3.62E-02**	7.18E-02	3.76E-02	4.62E-02	4.42E-02
	20.5	**3.28E-02**	8.88E-02	3.76E-02	5.11E-02	3.77E-02
3	**30.1**	**1.34E-01**	2.40E-01	2.52E-01	2.30E-01	1.87E-01
	30.2	**1.13E-01**	1.67E-01	2.27E-01	1.95E-01	1.46E-01
	30.3	**1.15E-01**	1.97E-01	2.27E-01	2.01E-01	1.47E-01
	40.1	**1.39E-01**	2.07E-01	2.79E-01	2.42E-01	1.56E-01
	40.2	**1.34E-01**	1.86E-01	2.67E-01	2.12E-01	1.80E-01
	40.3	**1.48E-01**	2.04E-01	2.62E-01	2.23E-01	1.54E-01
	50.1	**1.56E-01**	2.58E-01	3.30E-01	2.63E-01	1.70E-01
4	**50.1**	**2.58E-01**	2.99E-01	4.05E-01	3.31E-01	2.81E-01
	50.2	**2.60E-01**	3.45E-01	3.63E-01	2.97E-01	2.77E-01
	50.3	**2.64E-01**	3.69E-01	4.03E-01	3.01E-01	2.73E-01
KUNI						
2	**20.1**	**6.44E-02**	1.31E-01	9.19E-02	8.18E-02	6.97E-02
	20.2	**1.25E-01**	3.44E-01	2.24E-01	1.80E-01	1.26E-01
	20.3	**3.37E-02**	6.24E-02	4.79E-02	5.94E-02	3.98E-02
3	**30.1**	**1.63E-01**	2.48E-01	2.44E-01	2.47E-01	1.75E-01
	30.2	**2.15E-01**	2.98E-01	2.83E-01	3.06E-01	2.27E-01
	30.3	**1.30E-01**	1.88E-01	2.24E-01	2.14E-01	1.40E-01
	40.1	**1.88E-01**	2.69E-01	2.92E-01	2.83E-01	2.78E-01

Table 14 continued

q	n.ID	MO- MAHM$_E$	Without LA	With VO1	Without Velocity Operator	Without Cooperation
	40.2	**3.37E-01**	4.88E-01	5.38E-01	3.95E-01	3.40E-01
	40.3	**1.03E-01**	1.34E-01	2.22E-01	1.75E-01	1.07E-01
4	**50.1**	**2.89E-01**	3.08E-01	3.58E-01	3.05E-01	2.90E-01
	50.2	**7.42E-01**	9.00E-01	1.04E+00	8.18E-01	7.43E-01
	50.3	**1.31E-01**	1.64E-01	2.90E-01	2.49E-01	1.53E-01

Table 15 Mean *IGD* results for learning strategy, velocity operator, and cooperation

q	n.ID	MO- MAHM$_E$	Without LA	With VO1	Without velocity operator	Without cooperation
KRL						
2	**20.1**	**3.97E-03**	5.81E-03	4.43E-03	4.76E-03	4.30E-03
	20.2	**2.51E-03**	4.77E-03	2.84E-03	3.46E-03	2.54E-03
	20.3	**9.54E-04**	1.95E-03	1.25E-03	1.89E-03	9.77E-04
	20.4	**2.78E-03**	5.63E-03	3.06E-03	3.94E-03	3.95E-03
	20.5	**1.59E-03**	4.09E-03	2.33E-03	3.62E-03	1.72E-03
3	**30.1**	**1.48E-03**	3.33E-03	2.61E-03	2.45E-03	2.23E-03
	30.2	**1.33E-03**	2.50E-03	1.88E-03	1.71E-03	1.81E-03
	30.3	**1.50E-03**	2.62E-03	2.02E-03	1.88E-03	1.77E-03
	40.1	**2.03E-03**	3.20E-03	3.01E-03	2.72E-03	2.10E-03
	40.2	1.92E-03	3.00E-03	2.80E-03	2.26E-03	**1.86E-03**
	40.3	**2.05E-03**	2.74E-03	2.65E-03	2.38E-03	2.34E-03
	50.1	**2.09E-03**	3.67E-03	3.34E-03	2.88E-03	2.49E-03
4	**50.1**	**2.33E-03**	2.46E-03	3.05E-03	2.65E-03	2.91E-03
	50.2	**2.47E-03**	3.39E-03	3.00E-03	2.78E-03	2.78E-03
	50.3	**2.37E-03**	3.97E-03	3.39E-03	3.06E-03	2.64E-03
KUNI						
2	**20.1**	**4.11E-03**	8.36E-03	5.66E-03	5.43E-03	4.25E-03
	20.2	**5.76E-02**	9.02E-02	6.35E-02	7.36E-02	5.79E-02
	20.3	**1.34E-03**	2.65E-03	1.71E-03	2.25E-03	1.66E-03
3	**30.1**	**2.63E-03**	4.46E-03	3.32E-03	3.31E-03	2.69E-03
	30.2	**4.86E-03**	7.46E-03	5.50E-03	5.96E-03	4.87E-03
	30.3	**1.94E-03**	2.96E-03	2.72E-03	2.57E-03	1.96E-03
	40.1	3.11E-03	5.10E-03	3.87E-03	3.56E-03	**2.81E-03**
	40.2	**1.08E-02**	2.00E-02	2.07E-02	1.60E-02	1.09E-02
	40.3	**1.50E-03**	1.58E-03	2.04E-03	1.52E-03	1.66E-03
4	**50.1**	3.39E-03	4.09E-03	3.89E-03	3.62E-03	**2.41E-03**
	50.2	**3.21E-02**	7.19E-02	6.73E-02	4.53E-02	3.23E-02
	50.3	**8.39E-04**	1.24E-03	1.67E-03	1.46E-03	9.86E-04

Table 16 Mean *HV* and *IGD* results for *MO*-MAHM$_E$, *HH-LA*, and *HH-RILA*

q	n.ID	HV			IGD		
		MO- MAHM$_E$	HH-LA	HH-RILA	MO- MAHM$_E$	HH-LA	HH-RILA
KRL							
2	**20.1**	**5.92E-02**	2.06E-01	1.84E-01	**3.97E-03**	1.23E-02	1.06E-02
	20.2	**3.63E-02**	1.41E-01	1.97E-01	**2.51E-03**	8.38E-03	1.08E-02
	20.3	**2.25E-02**	1.25E-01	1.68E-01	**9.54E-04**	5.52E-03	6.93E-03
	20.4	**3.62E-02**	1.42E-01	1.79E-01	**2.78E-03**	9.13E-03	1.17E-02
	20.5	**3.28E-02**	1.38E-01	1.40E-01	**1.59E-03**	8.67E-03	7.30E-03
3	**30.1**	**1.34E-01**	3.58E-01	4.07E-01	**1.48E-03**	5.23E-03	6.00E-03
	30.2	**1.13E-01**	2.98E-01	3.05E-01	**1.33E-03**	4.38E-03	4.40E-03
	30.3	**1.15E-01**	3.58E-01	3.38E-01	**1.50E-03**	4.58E-03	4.24E-03
	40.1	**1.39E-01**	3.47E-01	3.36E-01	**2.03E-03**	4.79E-03	4.62E-03
	40.2	**1.34E-01**	3.35E-01	3.11E-01	**1.92E-03**	4.98E-03	4.90E-03
	40.3	**1.48E-01**	3.45E-01	3.32E-01	**2.05E-03**	5.07E-03	5.07E-03
	50.1	**1.56E-01**	3.63E-01	3.72E-01	**2.09E-03**	5.11E-03	5.25E-03
4	**50.1**	**2.58E-01**	4.67E-01	4.44E-01	**2.33E-03**	4.66E-03	4.43E-03
	50.2	**2.60E-01**	4.39E-01	3.95E-01	**2.47E-03**	5.07E-03	5.25E-03
	50.3	**2.64E-01**	4.27E-01	4.02E-01	**2.37E-03**	5.03E-03	5.08E-03
KUNI							
2	**20.1**	**6.44E-02**	2.47E-01	2.32E-01	**4.11E-03**	1.77E-02	1.50E-02
	20.2	**1.25E-01**	6.19E-01	5.73E-01	**5.76E-02**	1.39E-01	1.30E-01
	20.3	**3.37E-02**	1.45E-01	1.39E-01	**1.34E-03**	6.63E-03	6.41E-03
3	**30.1**	**1.63E-01**	4.22E-01	4.14E-01	**2.63E-03**	7.73E-03	7.49E-03
	30.2	**2.15E-01**	4.38E-01	5.23E-01	**4.86E-03**	9.63E-03	1.26E-02
	30.3	**1.30E-01**	3.40E-01	3.30E-01	**1.94E-03**	5.21E-03	4.58E-03
	40.1	**1.88E-01**	4.54E-01	4.72E-01	**3.11E-03**	8.59E-03	8.56E-03
	40.2	**3.37E-01**	7.72E-01	5.80E-01	**1.08E-02**	3.58E-02	2.29E-02
	40.3	**1.03E-01**	2.32E-01	2.31E-01	**1.50E-03**	2.80E-03	2.62E-03
4	**50.1**	**2.89E-01**	5.11E-01	4.87E-01	**3.39E-03**	5.66E-03	5.11E-03
	50.2	**7.42E-01**	1.31E+00	1.26E+00	**3.21E-02**	1.24E-01	1.11E-01
	50.3	**1.31E-01**	2.24E-01	2.34E-01	**8.39E-04**	1.55E-03	2.50E-03

Table 17 Mean *HV* and *IGD* results for *MO*-MAHM$_E$, *GRASP*, and *CoMOLD/D*

q	n.ID	HV			IGD		
		MO- MAHM$_E$	GRASP	CoMOLS/D	MO- MAHM$_E$	GRASP	CoMOLS/D
KRL							
2	**20.1**	**5.92E-02**	7.23E-02	1.99E-01	**3.97E-03**	4.86E-03	1.02E-02
	20.2	**3.63E-02**	5.93E-02	1.96E-01	**2.51E-03**	5.62E-03	8.93E-03
	20.3	**2.25E-02**	4.48E-02	1.29E-01	**9.54E-04**	2.02E-03	5.35E-03
	20.4	**3.62E-02**	7.11E-02	1.33E-01	**2.78E-03**	5.25E-03	8.83E-03
	20.5	**3.28E-02**	5.32E-02	1.71E-01	**1.59E-03**	3.79E-03	7.82E-03
3	**30.1**	**1.34E-01**	1.79E-01	1.41E-01	**1.48E-03**	2.22E-03	2.65E-03
	30.2	**1.13E-01**	1.67E-01	1.62E-01	**1.33E-03**	2.22E-03	2.94E-03
	30.3	1.15E-01	1.68E-01	**1.07E-01**	**1.50E-03**	1.95E-03	1.85E-03
	40.1	**1.39E-01**	2.30E-01	1.41E-01	**2.03E-03**	2.75E-03	2.45E-03
	40.2	1.34E-01	2.13E-01	**1.29E-01**	**1.92E-03**	2.62E-03	2.17E-03
	40.3	1.48E-01	2.00E-01	**1.43E-01**	**2.05E-03**	2.55E-03	2.53E-03
	50.1	**1.56E-01**	2.12E-01	1.61E-01	**2.09E-03**	2.68E-03	3.08E-03
4	**50.1**	2.58E-01	3.73E-01	**2.18E-01**	**2.33E-03**	3.23E-03	2.70E-03
	50.2	2.60E-01	3.58E-01	**1.94E-01**	2.47E-03	3.28E-03	**2.28E-03**
	50.3	2.64E-01	3.39E-01	**2.15E-01**	2.37E-03	3.00E-03	**2.23E-03**
KUNI							
2	**20.1**	**6.44E-02**	1.25E-01	2.17E-01	**4.11E-03**	8.11E-03	1.43E-02
	20.2	**1.25E-01**	3.26E-01	7.39E-01	**5.76E-02**	7.44E-02	1.77E-01
	20.3	**3.37E-02**	5.52E-02	1.18E-01	**1.34E-03**	1.84E-03	4.48E-03
3	**30.1**	**1.63E-01**	2.90E-01	2.01E-01	**2.63E-03**	4.51E-03	4.25E-03
	30.2	**2.15E-01**	5.16E-01	3.95E-01	**4.86E-03**	1.14E-02	8.60E-03
	30.3	**1.30E-01**	2.21E-01	1.46E-01	**1.94E-03**	3.40E-03	3.30E-03
	40.1	**1.88E-01**	3.50E-01	2.03E-01	**3.11E-03**	5.74E-03	4.51E-03
	40.2	**3.37E-01**	8.76E-01	6.86E-01	**1.08E-02**	4.15E-02	2.63E-02
	40.3	**1.03E-01**	1.71E-01	1.13E-01	**1.50E-03**	2.89E-03	2.79E-03
4	**50.1**	2.89E-01	4.12E-01	**2.03E-01**	3.39E-03	3.85E-03	**3.08E-03**
	50.2	**7.42E-01**	1.38E+00	1.25E+00	**3.21E-02**	1.80E-01	1.28E-01
	50.3	1.31E-01	1.65E-01	**1.26E-01**	**8.39E-04**	1.75E-03	2.94E-03

Table 18 Mean *HV* results for *MO*-MAHM$_E$, *MO-MAHM*, *HTA*, *HMOEA*, *HNSGA*, *MOTA*, and *NSTA*

q	n.ID	MO- MAHM$_E$	MO- MAHM	HTA	HMOEA	HNSGA	MOTA	NSTA
KRL								
2	**20.1**	5.92E-02	1.40E-01	**2.31E-02**	2.54E-01	1.26E-01	1.03E-01	9.61E-02
	20.2	**3.63E-02**	1.01E-01	4.22E-02	1.76E-01	1.30E-01	7.37E-02	5.94E-02
	20.3	**2.25E-02**	5.97E-02	5.78E-02	1.28E-01	9.10E-02	7.84E-02	6.78E-02
	20.4	**3.62E-02**	7.11E-02	7.04E-02	1.71E-01	9.07E-02	1.16E-01	1.01E-01
	20.5	**3.28E-02**	8.30E-02	5.31E-02	2.14E-01	1.32E-01	8.49E-02	7.91E-02
3	**30.1**	**1.34E-01**	3.40E-01	2.98E-01	2.51E-01	4.62E-01	3.23E-01	3.12E-01
	30.2	**1.13E-01**	2.61E-01	2.63E-01	2.96E-01	4.88E-01	2.87E-01	2.57E-01
	30.3	**1.15E-01**	2.98E-01	2.45E-01	2.29E-01	4.37E-01	2.84E-01	2.94E-01
	40.1	**1.39E-01**	3.72E-01	3.20E-01	2.50E-01	4.61E-01	3.52E-01	3.56E-01
	40.2	**1.34E-01**	3.87E-01	2.99E-01	2.57E-01	4.78E-01	3.33E-01	3.33E-01
	40.3	**1.48E-01**	3.61E-01	3.22E-01	2.55E-01	4.66E-01	3.43E-01	3.50E-01
	50.1	**1.56E-01**	4.71E-01	3.72E-01	3.00E-01	5.01E-01	3.94E-01	3.83E-01
4	**50.1**	**2.58E-01**	4.76E-01	4.33E-01	4.21E-01	6.54E-01	4.14E-01	4.23E-01
	50.2	**2.60E-01**	5.47E-01	3.84E-01	4.09E-01	6.33E-01	3.88E-01	3.94E-01
	50.3	**2.64E-01**	5.38E-01	3.90E-01	4.89E-01	6.80E-01	4.07E-01	4.14E-01
KUNI								
2	**20.1**	6.44E-02	1.15E-01	**5.42E-02**	2.40E-01	1.42E-01	7.63E-02	7.26E-02
	20.2	**1.25E-01**	3.79E-01	1.25E-01	6.56E-01	6.33E-01	3.59E-01	2.80E-01
	20.3	**3.37E-02**	8.84E-02	4.40E-02	1.41E-01	8.63E-02	5.84E-02	6.09E-02
3	**30.1**	**1.63E-01**	3.25E-01	2.31E-01	2.21E-01	4.60E-01	2.44E-01	2.88E-01
	30.2	2.15E-01	4.28E-01	**2.01E-01**	3.72E-01	4.00E-01	2.68E-01	2.80E-01
	30.3	**1.30E-01**	3.05E-01	1.96E-01	2.31E-01	4.86E-01	1.95E-01	2.60E-01
	40.1	**1.88E-01**	4.09E-01	2.45E-01	2.82E-01	5.32E-01	2.59E-01	2.84E-01
	40.2	3.37E-01	4.86E-01	**3.03E-01**	6.60E-01	6.49E-01	3.77E-01	3.18E-01
	40.3	**1.03E-01**	2.92E-01	1.93E-01	2.83E-01	5.02E-01	1.83E-01	2.41E-01
4	**50.1**	**2.89E-01**	5.56E-01	3.47E-01	5.28E-01	7.97E-01	2.98E-01	3.83E-01
	50.2	7.42E-01	6.50E-01	5.85E-01	1.27E+00	1.24E+00	6.70E-01	**5.65E-01**
	50.3	**1.31E-01**	4.25E-01	3.18E-01	3.63E-01	4.31E-01	3.21E-01	3.09E-01

Table 19 Mean *IGD* results for *MO*-MAHM$_E$, *MO-MAHM*, *HTA*, *HMOEA*, *HNSGA*, *MOTA*, and *NSTA*

q	n.ID	MO- MAHM$_E$	MO- MAHM	HTA	HMOEA	HNSGA	MOTA	NSTA
KRL								
2	**20.1**	3.97E-03	9.10E-03	**2.81E-03**	1.27E-02	7.43E-03	7.46E-03	7.49E-03
	20.2	**2.51E-03**	5.36E-03	3.80E-03	7.84E-03	5.69E-03	5.38E-03	4.55E-03
	20.3	**9.54E-04**	3.40E-03	3.46E-03	5.57E-03	4.94E-03	4.41E-03	4.22E-03
	20.4	**2.78E-03**	4.94E-03	7.97E-03	1.03E-02	6.24E-03	1.02E-02	9.31E-03
	20.5	**1.59E-03**	4.47E-03	2.79E-03	1.02E-02	6.66E-03	6.31E-03	6.75E-03
3	**30.1**	**1.48E-03**	3.78E-03	2.05E-03	2.02E-03	3.98E-03	2.19E-03	2.21E-03
	30.2	**1.33E-03**	2.68E-03	1.75E-03	2.35E-03	4.27E-03	1.89E-03	1.98E-03
	30.3	1.50E-03	2.89E-03	**1.47E-03**	1.99E-03	4.68E-03	1.70E-03	1.97E-03
	40.1	**2.03E-03**	4.65E-03	2.69E-03	2.77E-03	5.31E-03	3.18E-03	3.18E-03
	40.2	**1.92E-03**	4.42E-03	2.34E-03	2.62E-03	5.37E-03	2.90E-03	2.83E-03
	40.3	**2.05E-03**	4.46E-03	2.55E-03	3.28E-03	6.02E-03	2.95E-03	2.97E-03
	50.1	**2.09E-03**	5.87E-03	2.89E-03	3.40E-03	5.62E-03	3.47E-03	3.37E-03
4	**50.1**	**2.33E-03**	4.21E-03	2.37E-03	2.99E-03	4.61E-03	2.68E-03	2.40E-03
	50.2	**2.47E-03**	5.68E-03	2.51E-03	3.01E-03	4.92E-03	2.85E-03	2.65E-03
	50.3	2.37E-03	5.53E-03	**2.34E-03**	3.53E-03	5.41E-03	2.74E-03	2.72E-03
KUNI								
2	**20.1**	4.11E-03	7.07E-03	**3.74E-03**	1.60E-02	8.75E-03	4.45E-03	4.33E-03
	20.2	5.76E-02	1.00E-01	**4.97E-02**	1.48E-01	1.45E-01	8.38E-02	6.55E-02
	20.3	**1.34E-03**	4.13E-03	1.93E-03	5.69E-03	4.03E-03	2.60E-03	2.70E-03
3	**30.1**	2.63E-03	4.51E-03	**2.55E-03**	2.83E-03	6.68E-03	2.67E-03	4.06E-03
	30.2	4.86E-03	1.03E-02	**4.31E-03**	8.40E-03	9.45E-03	5.20E-03	6.83E-03
	30.3	1.94E-03	3.78E-03	**1.70E-03**	2.36E-03	6.06E-03	1.80E-03	3.01E-03
	40.1	3.11E-03	5.88E-03	**2.77E-03**	3.98E-03	7.97E-03	3.04E-03	4.09E-03
	40.2	1.08E-02	2.42E-02	**9.28E-03**	2.55E-02	2.62E-02	1.11E-02	1.07E-02
	40.3	**1.50E-03**	2.82E-03	1.79E-03	2.98E-03	9.18E-03	1.79E-03	2.33E-03
4	**50.1**	3.39E-03	5.94E-03	**3.21E-03**	4.80E-03	8.03E-03	3.48E-03	3.86E-03
	50.2	3.21E-02	5.32E-02	**2.33E-02**	1.15E-01	1.01E-01	2.76E-02	2.35E-02
	50.3	**8.39E-04**	2.54E-03	1.67E-03	1.86E-03	3.60E-03	1.79E-03	1.55E-03

Acknowledgements This research was partially supported by the Program for Young Researchers of the Federal University of Bahia (PROPCI-PROPG/UFBA 007/2022 - JOVEMPESQ), High-Performance Computing Center at Federal University do Rio Grande do Norte (NPAD/UFRN), and National Council for Scientific and Technological Development (CNPq), Brazil, under grant 310088/2020-8 and 402842/2023-5.

References

1. Almeida CP, Gonçalves RA, Goldbarg EFG, Goldbarg MC, Delgado MR (2014) Transgenetic algorithms for the multi-objective quadratic assignment problem. In: Brazilian Conference on Intelligent Systems. BRACIS 2014, pp. 312–317. IEEE
2. Behmanesh R, Rahimi I, Gandomi AH (2021) Evolutionary many-objective algorithms for combinatorial optimization problems: a comparative study. Archiv Comput Method Eng. https://doi.org/10.1007/s11831-020-09415-3
3. Bleuler S, Laumanns M, Thiele L, Zitzler E (2003) PISA - a platform and programming language independent interface for search algorithms. In: Fonseca CM, Fleming PJ, Zitzler E, Deb K, Thiele L (eds) Evolutionary Multi-Criterion Optimization. Springer, Berlin, pp 494–508
4. Blum C, Puchinger J, Raidl GR, Roli A (2011) Hybrid metaheuristics in combinatorial optimization: a survey. Appl Soft Comput 11(6):4135–4151. https://doi.org/10.1016/j.asoc.2011.02.032
5. Cai X, Hu M, Gong D, Guo Yn, Zhang Y, Fan Z, Huang Y (2019) A decomposition-based coevolutionary multiobjective local search for combinatorial multiobjective optimization. Swarm and Evolut Comput 49:178–193. https://doi.org/10.1016/j.swevo.2019.05.007
6. Chen J, Ding J, Tan KC, Chen Q (2021) A decomposition-based evolutionary algorithm for scalable multi/many-objective optimization. Memetic Comput 13:413–432. https://doi.org/10.1007/s12293-021-00330-z
7. Coello CAC, Sierra MR (2004) A study of the parallelization of a coevolutionary multi-objective evolutionary algorithm. In: R. Monroy, G. Arroyo-Figueroa, L. E. Sucar, and J. H. S. Azuela (Eds.), Mexican International Conference on Artificial Intelligence. In: MICAI 2004, Volume 2972 of Lecture Notes in Computer Science, pp. 688–697. Springer
8. Conover WJ (1999) Practical Nonparametric Statistics, vol 3. Wiley, New York, USA
9. Deb K, Jain H (2014) An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints. IEEE Trans Evolut Comput 18(4):577–601. https://doi.org/10.1109/TEVC.2013.2281535
10. Deb K, Pratap A, Agarwal S, Meyarivan T (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Trans Evolut Comput 6(2):182–197. https://doi.org/10.1109/4235.996017
11. Felten F, Talbi EG, Danoy G (2024) Multi-objective reinforcement learning based on decomposition: A taxonomy and framework. J Artif Int Res. https://doi.org/10.1613/jair.1.15702
12. Fernandes IFC, Goldbarg EFG, Maia SMDM, Goldbarg MC (2021) Multi- and many-objective path-relinking: a taxonomy and decomposition approach. Comput Operat Res 133:105370. https://doi.org/10.1016/j.cor.2021.105370
13. Fernandes IFC, Silva IRM, Goldbarg EFG, Maia SMDM, Goldbarg MC (2020) A PSO-inspired architecture to hybridise multi-objective metaheuristics. Memetic Comput 12(3):235–249. https://doi.org/10.1007/s12293-020-00307-4
14. Glover F (1997) Tabu search and adaptive memory programming-advances, applications and challenges, In: Interfaces in Computer Science and Operations Research, eds. Barr, R., R. Helgason, and J. Kennington, Volume 7 of Operations Research/Computer Science Interfaces Series, 1–75. Boston, MA: Springer. https://doi.org/10.1007/978-1-4615-4102-8_1
15. Goldbarg EFG, Goldbarg MC (2009) Transgenetic algorithm: A new endosymbiotic approach for evolutionary algorithms, In: Foundations of Computational Intelligence - Volume 3: Global Optimization, eds. Abraham, A., A.E. Hassanien, P. Siarry, and A.P. Engelbrecht, Volume 203 of Studies in Computational Intelligence, 425–460. Berlin, Heidelberg: Springer. https://doi.org/10.1007/978-3-642-01085-9_14
16. Guo X (2022) A survey of decomposition based evolutionary algorithms for many-objective optimization problems. IEEE Access 10:72825–72838. https://doi.org/10.1109/ACCESS.2022.3188762
17. Hayes CF, Rădulescu R, Bargiacchi E, Källström J, Macfarlane M, Reymond M, Verstraeten T, Zintgraf LM, Dazeley R, Heintz F et al (2022) A practical guide to multi-objective reinforcement learning and planning. Auto Agents and Multi-Agent Syst 36(1):26
18. Hu C, Wang Q, Gong W, Yan X (2022) Multi-objective deep reinforcement learning for emergency scheduling in a water distribution network. Memetic Comput 14(2):211–223. https://doi.org/10.1007/s12293-022-00366-9
19. Jain H, Deb K (2014) An evolutionary many-objective optimization algorithm using reference-point based nondominated sorting approach, part II: Handling constraints and extending to an adaptive approach. IEEE Trans Evolut Comput 18(4):602–622. https://doi.org/10.1109/TEVC.2013.2281534
20. Jaszkiewicz A (2018) Many-objective Pareto local search. Euro J Operat Res 271(3):1001–1013. https://doi.org/10.1016/j.ejor.2018.06.009
21. Karami F, Dariane AB (2022) A review and evaluation of multi and many-objective optimization: methods and algorithms. Glob J Ecol 7(2):104–119
22. Knowles J, Corne D (2003) Instance generators and test suites for the multiobjective quadratic assignment problem. In: C. M. Fonseca, P. J. Fleming, E. Zitzler, L. Thiele, and K. Deb (Eds.), Evolutionary Multi-Criterion Optimization. EMO 2003, Volume 2632 of Lecture Notes in Computer Science, Berlin, Heidelberg, pp. 295–310. Springer
23. Knowles JD (2002) Local-search and Hybrid Evolutionary Algorithms for Pareto Optimization. Ph. D. thesis, University of Reading UK
24. Knowles JD, Thiele L, Zitzler E (2005) A tutorial on the performance assessment of stochastic multiobjective optimizers. TIK-Report 214
25. Koopmans TC, Beckmann MJ (1957) Assignment problems and the location of economic activities. Econometrica 25(1):53–76. https://doi.org/10.2307/1907742
26. Kouka N, BenSaid F, Fdhila R, Fourati R, Hussain A, Alimi AM (2023) A novel approach of many-objective particle swarm optimization with cooperative agents based on an inverted generational distance indicator. Inform Sci 623:220–241
27. Li B, Li J, Tang K, Yao X (2015) Many-objective evolutionary algorithms: a survey. ACM Comput Surv 48(1):13:1-13:35. https://doi.org/10.1145/2792984
28. Li H, Landa-Silva D (2009) An elitist GRASP metaheuristic for the multi-objective quadratic assignment problem. In: M. Ehrgott, C. M. Fonseca, X. Gandibleux, J. Hao, and M. Sevaux (Eds.), Evolutionary Multi-Criterion Optimization. EMO 2009, Volume 5467 of Lecture Notes in Computer Science, Berlin, Heidelberg, pp. 481–494. Springer
29. Li W, Özcan E, John R (2019) A learning automata-based multiobjective hyper-heuristic. IEEE Trans Evolut Comput 23(1):59–73. https://doi.org/10.1109/TEVC.2017.2785346

30. Liu SC, Zhan ZH, Tan KC, Zhang J (2021) A multiobjective framework for many-objective optimization. IEEE Trans Cybernet 52(12):13654–13668
31. López-Ibáñez M, Dubois-Lacoste J, Cáceres LP, Birattari M, Stützle T (2016) The irace package: iterated racing for automatic algorithm configuration. Operat Res Perspect 3:43–58. https://doi.org/10.1016/j.orp.2016.09.002
32. Machado MO, Fernandes IFC, Maia SMDM, Goldbarg EFG (2024) A hybrid multi-agent metaheuristic for the offshore wind farm cable routing problem. Expert Syst Appl 255:124668
33. Mao Z, Liu M (2022) A local search-based many-objective five-element cycle optimization algorithm. Swarm and Evolut Comput 68:101009
34. Martí R, Campos V, Resende MG, Duarte A (2015) Multiobjective GRASP with path relinking. Euro J Operat Res 240(1):54–71. https://doi.org/10.1016/j.ejor.2014.06.042
35. Morteza H, Jameii SM, Sohrabi MK (2023) An improved learning automata based multi-objective whale optimization approach for multi-objective portfolio optimization in financial markets. Expert Syst Appl 224:119970
36. Murata T, Ishibuchi H, Gen M (2001) Specification of genetic search directions in cellular multi-objective genetic algorithms. In: E. Zitzler, L. Thiele, K. Deb, C. A. C. Coello, and D. Corne (Eds.), Evolutionary Multi-Criterion Optimization. EMO 2001, Volume 1993 of Lecture Notes in Computer Science, Berlin, Heidelberg, pp. 82–95. Springer
37. Murata T, Ishibuchi H, Tanaka H (1996) Genetic algorithms for flowshop scheduling problems. Comput Indust Eng 30(4):1061–1071. https://doi.org/10.1016/0360-8352(96)00053-8
38. Oliveira ALC, Britto A, Gusmão R (2023) Machine learning enhancing metaheuristics: a systematic review. Soft Comput 27:1–28. https://doi.org/10.1007/s00500-023-08886-3
39. Qin S, Sun C, Zhang G, He X, Tan Y (2020) A modified particle swarm optimization based on decomposition with different ideal points for many-objective optimization problems. Complex Intell Syst 6:263–274. https://doi.org/10.1007/s40747-020-00134-7
40. Reddy S, Dulikravich GS (2022) A self-adapting algorithm for many-objective optimization. Appl Soft Comput 129:109484
41. Ribeiro CC, Resende MGC (2012) Path-relinking intensification methods for stochastic local search algorithms. J Heuristics 18(2):193–214. https://doi.org/10.1007/s10732-011-9167-1
42. Sahni S, Gonzalez T (1976) P-complete approximation problems. J ACM 23(3):555–565. https://doi.org/10.1145/321958.321975
43. Silva IRM, Goldbarg EFG, de Carvalho EB, Goldbarg MC (2017) Mo-mahm: A parallel multi-agent architecture for hybridization of metaheuristics for multi-objective problems. In: 2017 IEEE Congress on Evolutionary Computation (CEC), pp. 580–587. IEEE
44. Silva MAL, de Souza SR, Souza MJF, França Filho MF (2018) Hybrid metaheuristics and multi-agent systems for solving optimization problems: a review of frameworks and a comparative analysis. Appl Soft Comput 71:433–459. https://doi.org/10.1016/j.asoc.2018.06.050
45. Taillard É (1991) Robust taboo search for the quadratic assignment problem. Parallel Comput 17(4–5):443–455. https://doi.org/10.1016/S0167-8191(05)80147-4
46. Talbi EG (2015) Hybrid metaheuristics for multi-objective optimization. J Algorithms Comput Technol 9(1):41–63. https://doi.org/10.1260/1748-3018.9.1.41
47. Tang W, Liu H, Chen L, Tan KC, Cheung Y (2020) Fast hypervolume approximation scheme based on a segmentation strategy. Inform Sci 509:320–342. https://doi.org/10.1016/j.ins.2019.02.054
48. Uribe NR, Herrán A, Colmenar JM (2024) A GRASP method for the Bi-objective multiple row equal facility layout problem. Appl Soft Comput 163:111897. https://doi.org/10.1016/j.asoc.2024.111897
49. Wang H, Wang S, Wei Z, Zeng T, Ye T (2023) An improved many-objective artificial bee colony algorithm for cascade reservoir operation. Neural Comput Appl 35(18):13613–13629
50. Xu J, Li K, Abusara M (2022) Preference based multi-objective reinforcement learning for multi-microgrid system optimization problem in smart grid. Memetic Comput 14(2):225–235. https://doi.org/10.1007/s12293-022-00357-w
51. Yang L, Cao J, Li K, Zhang Y, Xu R, Li K (2024) A many-objective evolutionary algorithm based on interaction force and hybrid optimization mechanism. Swarm and Evolut Comput 90:101667
52. Yuan Y, Xu H, Wang B, Zhang B, Yao X (2016) Balancing convergence and diversity in decomposition-based many-objective optimizers. IEEE Trans Evolut Comput 20(2):180–198. https://doi.org/10.1109/TEVC.2015.2443001
53. Zhang Q, Li H (2007) MOEA/D: a multiobjective evolutionary algorithm based on decomposition. IEEE Trans Evolut Comput 11(6):712–731. https://doi.org/10.1109/TEVC.2007.892759
54. Zhang Y, Zhou K, Tang J (2024) Harnessing collaborative learning automata to guide multi-objective optimization based inverse analysis for structural damage identification. Appl Soft Comput 160:111697. https://doi.org/10.1016/j.asoc.2024.111697
55. Zhao H, Zhang C (2020) A decomposition-based many-objective artificial bee colony algorithm with reinforcement learning. Appl Soft Comput 86:105879. https://doi.org/10.1016/j.asoc.2019.105879
56. Zhao H, Zhang C (2020) An online-learning-based evolutionary many-objective algorithm. Inform Sci 509:1–21. https://doi.org/10.1016/j.ins.2019.08.069
57. Zhao H, Zhang C, Ning J, Zhang B, Sun P, Feng Y (2019) A comparative study of the evolutionary many-objective algorithms. Progress in Artific Intell 8(1):15–43. https://doi.org/10.1007/s13748-019-00174-2
58. Zitzler E, Thiele L (1998) Multiobjective optimization using evolutionary algorithms - a comparative case study. In: A. E. Eiben, T. Bäck, M. Schoenauer, and H. Schwefel (Eds.), Parallel Problem Solving from Nature. PPSN 1998, Volume 1498 of Lecture Notes in Computer Science, Berlin, Heidelberg, pp. 292–301. Springer